Riassunti e Appunti di Natural Language Processing

Appunti di corso



Università di Catania

Università degli Studi di Catania
Dipartimento di Matematica e Informatica
Corso di Laurea Magistrale in Informatica (LM-18)

Autore
Emanuele Galiano

Anno accademico
Anno Accademico 2026/2027

Versione: 4 novembre 2026

Prefazione

Queste dispense sono nate come appunti personali per il corso di *Natural Language Processing* tenuto presso il Dipartimento di Matematica e Informatica dell'Università di Catania dal Prof. **Misael Mongiovì** nell'anno accademico 2026/2027. L'obiettivo principale di questo materiale è stato fornire una risorsa personale di studio sulla parte teorica del corso, raggruppando concetti chiave, definizioni, immagini ed esempi in un unico documento, e integrando le slide fornite dal docente con il testo di riferimento del corso, *Speech and Language Processing* di D. Jurafsky e J. H. Martin [11].

Questo file non deve essere visto come un testo ufficiale o completo sull'argomento, in quanto potrebbero esserci errori, omissioni o imprecisioni. Invito pertanto chi legge questo documento a consultare le dispense ufficiali del corso proposte dal docente ed eventuali testi di riferimento consigliati. Inoltre, invito chiunque noti errori o abbia suggerimenti a contattarmi via email:

- **Email personale**: galianoo.emanuele@gmail.com,
- **Email universitaria**: emanuele.galiano@studium.unict.it;

oppure, se ancora presente online, ad aprire una **issue** o una **pull request** nel repository GitHub associato a queste dispense:

`https://github.com/emanuelegaliano/Natural-Language-Processing`

In particolare, all'interno del repository GitHub è presente il codice open-source dei sorgenti LaTeX utilizzati per generare queste dispense, oltre a tutti i file di supporto come immagini e codici per la generazione di alcuni grafici presenti nel testo. Le figure interattive sono disponibili solo nella versione web delle dispense: nel PDF ciascuna di esse è sostituita da una figura statica.

Licenza

Questo documento, intitolato *Riassunti e Appunti di Natural Language Processing – Appunti di corso*, è distribuito sotto licenza **Creative Commons Attribution–ShareAlike 4.0 International (CC BY-SA 4.0)**.

La presente licenza è stata scelta con l'obiettivo di favorire la diffusione, la condivisione e il riutilizzo del materiale didattico, garantendo al contempo il riconoscimento degli autori originali e la preservazione della natura open delle opere derivate.

Autori: Emanuele Galiano
Affiliazione: Università degli Studi di Catania – Dipartimento di Matematica e Informatica
Anno accademico: Anno Accademico 2026/2027

Diritti concessi

In conformità con i termini della licenza Creative Commons BY-SA 4.0, è consentito a chiunque di:

- copiare e ridistribuire il materiale in qualsiasi mezzo o formato;
- adattare, modificare e trasformare il materiale;
- utilizzare il materiale anche per scopi commerciali.

Tali diritti sono concessi a titolo gratuito e non possono essere revocati, purché siano rispettate le condizioni indicate nella sezione seguente.

Condizioni

L'utilizzo del materiale è subordinato al rispetto delle seguenti condizioni:

- **Attribuzione (BY):** deve essere fornita un'adeguata attribuzione dell'opera, citando **titolo**, **autori** e, ove possibile, la **fonte**. L'attribuzione deve essere effettuata in modo ragionevole

e non tale da suggerire che gli autori originali approvino l'uso o le modifiche apportate.

- **Condividi allo stesso modo (SA):** nel caso in cui il materiale venga modificato, trasformato o utilizzato per creare opere derivate, tali opere devono essere distribuite sotto la *stessa licenza* Creative Commons Attribution–ShareAlike 4.0 International.

Non è consentito applicare termini legali o misure tecnologiche che limitino giuridicamente altri utenti dall'esercitare i diritti concessi dalla licenza.

Assenza di garanzia

Il materiale è fornito *"così com'è"*, senza garanzie di alcun tipo, esplicite o implicite. In particolare, gli autori non garantiscono l'accuratezza, la completezza o l'assenza di errori nel contenuto del documento e declinano ogni responsabilità per eventuali danni derivanti dall'uso del materiale.

Testo completo della licenza

Il testo legale completo della licenza Creative Commons Attribution–ShareAlike 4.0 International è disponibile al seguente indirizzo:

`https://creativecommons.org/licenses/by-sa/4.0/`

Copyright © 2026
Emanuele Galiano

Indice

Licenza **v**
Diritti concessi . . . v
Condizioni . . . v
Assenza di garanzia . . . vi
Testo completo della licenza . . . vi

1 Introduzione **1**
1.1 Perché il Natural Language Processing . . . 1
1.2 La pipeline classica . . . 2
1.2.1 Analisi sintattica . . . 3
1.2.2 Rappresentazione del significato . . . 3
1.3 Il significato come vettore . . . 4
1.4 Large Language Models . . . 5
1.5 Espressioni regolari . . . 6
1.5.1 Disgiunzioni . . . 7
1.5.2 Contatori e caratteri jolly . . . 7
1.5.3 Backslash e ancore . . . 8
1.5.4 Un esempio: trovare *the* . . . 8
1.5.5 Sostituzioni e capture group . . . 9
1.5.6 Lookahead . . . 10
1.6 ELIZA . . . 10

I La pipeline classica **13**

2 Normalizzazione del testo **15**
2.1 Parole e corpora . . . 15
2.1.1 Corpora . . . 16
2.2 Tokenizzazione delle parole . . . 16
2.2.1 Tokenizzare con gli strumenti UNIX . . . 17
2.2.2 Problemi della tokenizzazione . . . 18
2.2.3 Tokenizzare con NLTK . . . 18
2.2.4 Lingue senza spazi . . . 19
2.3 Tokenizzazione in sottoparole . . . 19
2.3.1 Il token learner di BPE . . . 20
2.3.2 Il token segmenter di BPE . . . 21

2.3.3 Proprietà dei token BPE . . . 21
2.4 Normalizzazione delle parole . . . 21
2.4.1 Case folding . . . 22
2.4.2 Lemmatizzazione . . . 22
2.4.3 Stemming . . . 22
2.4.4 Lemmatizzare con spaCy . . . 23
2.5 Segmentazione delle frasi . . . 23

3 Modelli di linguaggio N-gram 25
3.1 Modelli di linguaggio probabilistici . . . 25
3.2 L'assunzione di Markov . . . 26
3.3 Stimare le probabilità . . . 27
3.3.1 Il Berkeley Restaurant Project . . . 28
3.3.2 Problemi pratici . . . 29
3.4 Valutare un modello di linguaggio . . . 30
3.4.1 Perplexity . . . 30
3.5 Generalizzazione e zeri . . . 31
3.5.1 Il metodo di visualizzazione di Shannon . . . 31
3.5.2 Il pericolo dell'overfitting . . . 32
3.6 Smoothing . . . 33
3.6.1 Laplace smoothing . . . 33
3.6.2 Backoff e interpolazione . . . 34
3.6.3 N-gram su scala web . . . 35
3.6.4 Absolute discounting . . . 35
3.6.5 Kneser-Ney smoothing . . . 36
3.6.6 Modelli di linguaggio avanzati . . . 37

4 Naive Bayes e classificazione del testo 39
4.1 Classificazione del testo . . . 40
4.2 Il classificatore Naive Bayes . . . 40
4.2.1 La regola di Bayes applicata ai documenti . . . 41
4.2.2 Le assunzioni di Naive Bayes . . . 41
4.3 Addestrare Naive Bayes . . . 42
4.4 Un esempio svolto . . . 44
4.5 Ottimizzare Naive Bayes per la sentiment analysis . . . 45
4.5.1 Binary Naive Bayes . . . 45
4.5.2 Negazione . . . 46
4.5.3 Lessici . . . 46
4.6 Naive Bayes per altri task . . . 46
4.7 Naive Bayes come modello di linguaggio . . . 47
4.8 Valutazione . . . 47
4.8.1 Più di due classi . . . 49
4.9 Development set e cross-validation . . . 50
4.10 Significatività statistica . . . 50
4.10.1 Test di ipotesi . . . 51

4.10.2 Il paired bootstrap test . . . 51
4.11 Danni nei classificatori . . . 53

5 Regressione logistica 55
5.1 Classificatori generativi e discriminativi . . . 55
5.2 Machine learning supervisionato . . . 55
5.2.1 Le componenti di un classificatore probabilistico . . . 57
5.3 Classificazione con la regressione logistica . . . 57
5.3.1 La sigmoide . . . 58
5.3.2 Un esempio di sentiment analysis . . . 59
5.4 La cross-entropy loss . . . 60
5.5 Discesa del gradiente . . . 61
5.5.1 Il gradiente della regressione logistica . . . 63
5.5.2 La discesa del gradiente stocastica . . . 63
5.5.3 Mini-batch . . . 64
5.6 Overfitting e regolarizzazione . . . 64
5.7 Regressione logistica multinomiale . . . 65
5.7.1 Il softmax . . . 65
5.7.2 Il softmax nella regressione logistica multinomiale . . . 66
5.8 La classificazione in pratica . . . 67

Bibliografia 69

Capitolo 1

Introduzione

Il **Natural Language Processing** (NLP) è il punto d'incontro tra il linguaggio umano e i sistemi computazionali: è l'insieme delle tecniche che permettono a una macchina di

- **capire** il linguaggio umano, scritto o parlato;
- **interpretare** sfumature, contesto e significati;
- **generare** testi coerenti e pertinenti;
- **interagire** attraverso interfacce in linguaggio naturale.

Il problema è che il linguaggio è ambiguo, vago e dipende dal contesto. Trasformare il senso di una frase in una rappresentazione che un computer possa elaborare è tutt'altro che semplice, e buona parte del corso riguarda proprio i diversi modi in cui si è provato a farlo.

In questo capitolo diamo una panoramica degli argomenti del corso, dalla pipeline classica ai Large Language Models, e poi vediamo il primo strumento concreto per lavorare con il testo: le **espressioni regolari**, con cui costruiremo anche un piccolo chatbot (ELIZA).

1.1 Perché il Natural Language Processing

Padroneggiare il linguaggio non serve solo a conversare. Le motivazioni sono almeno quattro:

- **Comunicare con l'umano**: interagire nella lingua di tutti i giorni invece che con comandi rigidi o codice, come fanno assistenti vocali, chatbot e motori di ricerca conversazionali. In altre parole, parliamo ai computer invece di programmarli.
- **Pensiero**: il linguaggio è il mezzo con cui esprimiamo e organizziamo il ragionamento, e gran parte della conoscenza umana è scritta. I modelli più recenti "ragionano" proprio generando testo, scomponendo un problema in passi (*chain of thought*).
- **Unificare testo, immagini e azioni**: il linguaggio fa da collante tra modalità diverse. Descrive immagini, trascrive e sintetizza il parlato, si traduce in istruzioni per sistemi autonomi.
- **Sistemi agentici**: un modello che comprende e produce linguaggio può pianificare, scegliere strumenti e agire per raggiungere un obiettivo.

L'ultimo punto merita un esempio. Il loop **ReAct** [28] alterna tre fasi (figura 1.1). Nel **pensiero** il modello scrive, in linguaggio naturale, cosa sa e cosa gli manca. Nell'**azione** sceglie uno strumento e lo invoca (cerca, calcola, legge un documento). Nell'**osservazione** il risultato dello strumento rientra nel testo come nuova informazione. Il ciclo si ripete finché l'obiettivo non è raggiunto, e solo allora il modello produce la risposta finale. La differenza con un chatbot è quindi questa: un chatbot risponde una volta, mentre un agente persegue un obiettivo passo dopo passo, correggendo la rotta con ciò che osserva.

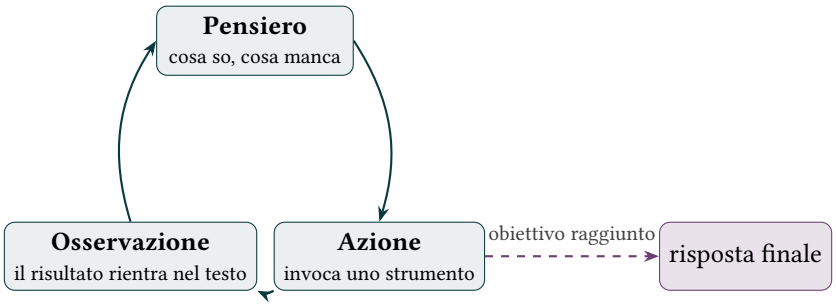


Figura 1.1. Il loop ReAct: il modello alterna pensiero, azione e osservazione finché l'obiettivo non è raggiunto, e solo allora produce la risposta finale.

1.2 La pipeline classica

Per decenni l'idea guida dell'NLP è stata la **pipeline**: il testo attraversa una sequenza di moduli separati, e ognuno produce una struttura esplicita e ispezionabile che diventa l'input del modulo successivo (figura 1.2). Consideriamo la frase "Ho visto l'elefante con il cannocchiale", che torneremo a usare più volte. I moduli sono:

- **Normalizzazione**: tokenizzazione ed espressioni regolari (capitolo 2).
- **POS tagging e NER**: categorie grammaticali delle parole ed entità nominate.
- **Parsing sintattico**: la struttura della frase, sotto forma di albero.
- **Parsing semantico**: una rappresentazione formale del significato.
- **Ragionamento e risposta**: inferenza, risposta alla richiesta, azione.

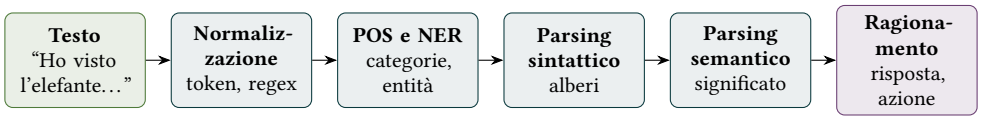


Figura 1.2. La pipeline classica: ogni modulo è separato dagli altri e produce una struttura esplicita, che diventa l'input del modulo successivo.

Ogni modulo può essere realizzato con **regole scritte a mano** oppure con **modelli appresi da dati annotati**, ad esempio da un *treebank* (una raccolta di frasi annotate con il loro albero sintattico). Accanto ai moduli ci sono poi degli strumenti statistici trasversali, che vedremo

nei prossimi capitoli: i **modelli di linguaggio N-gram**, che assegnano una probabilità a una sequenza di parole (capitolo 3), e i **classificatori** come Naive Bayes e la regressione logistica (capitoli 4 e 5).

La pipeline ha però dei limiti evidenti: è fragile (un errore in un modulo si propaga a tutti i successivi), ha una copertura limitata, non ha conoscenza del mondo, e il numero di interpretazioni possibili esplode con l'ambiguità. Nella frase dell'elefante, ad esempio, il cannocchiale può essere lo strumento con cui ho visto l'elefante, oppure qualcosa che l'elefante ha con sé. Sono questi limiti ad aver portato alla svolta dei modelli appresi da grandi quantità di testo.

1.2.1 Analisi sintattica

Il parsing sintattico restituisce la struttura della frase. Ne esistono due rappresentazioni classiche, entrambe in figura 1.3. L'**albero a costituenti** raggruppa le parole in sintagmi annidati, mentre l'**albero a dipendenze** collega direttamente le parole tra loro con archi etichettati: *flight* è l'oggetto diretto (`dobj`) di *prefer*, *I* ne è il soggetto (`nsubj`), e così via.

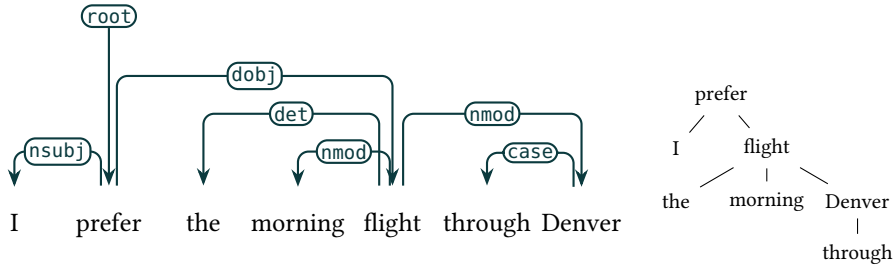


Figura 1.3. Le due rappresentazioni sintattiche della frase "I prefer the morning flight through Denver": a sinistra l'albero a dipendenze, con un arco etichettato per ogni relazione grammaticale, a destra la stessa struttura disegnata come albero, con ogni parola sotto quella da cui dipende.

1.2.2 Rappresentazione del significato

Il parsing semantico va un passo oltre e produce una rappresentazione formale del significato. La più classica è la **logica del primo ordine**. La frase "Ho un'auto", ad esempio, diventa:

$$\exists e, y \;\; \text{Avere}(e) \wedge \text{Possessore}(e, \text{Parlante}) \wedge \text{Posseduto}(e, y) \wedge \text{Auto}(y)$$

ovvero: esiste una situazione (o evento) e di "avere", il cui possessore è chi parla e la cui cosa posseduta y è un'auto.

Una volta che la frase è una formula logica, la macchina può fare quello che sa fare meglio: verificare, dedurre e rispondere a interrogazioni in modo esatto e spiegabile. Ad esempio, "A Matteo piace il ristorante Frasca" diventa $\text{Piace}(\text{matteo}, \text{frasca})$, e per verificarla basta controllare che la coppia $\langle \text{matteo}, \text{frasca} \rangle$ compaia nella relazione $\text{Piace} = \{\langle \text{matteo}, \text{frasca} \rangle, \langle \text{katia}, \text{med} \rangle, \dots\}$ della base di conoscenza: se c'è, la frase è vera in quel "modello" del mondo.

Ruoli semantici. Un modo per standardizzare i predicati è quello dei **ruoli semantici**: qualunque sia il verbo, le domande sono sempre le stesse. Chi agisce (**agente**)? Cosa subisce l'azione

(**tema**)? Con cosa (**strumento**)? In "Sasha ha rotto la finestra" l'agente è Sasha e il tema la finestra, in "Pat ha aperto la porta" l'agente è Pat e il tema la porta.

Risorse come *PropBank* e *FrameNet* catalogano a mano i ruoli di migliaia di verbi, e la macchina impara a etichettarli in una frase nuova (*Semantic Role Labeling*). Lo schema è lo stesso del treebank: non si scrivono regole, si annotano esempi e si impara a imitarli.

Grafi di significato. L'**Abstract Meaning Representation** (AMR) [1] standardizza invece i concetti, rappresentando il significato come un grafo (figura 1.4). In "The boy wants the girl to believe him" c'è un evento di "volere" (`want-01`): chi vuole (`ARG0`) è un ragazzo, e ciò che vuole (`ARG1`) è un evento di "credere" (`believe-01`) in cui la ragazza crede allo stesso ragazzo. La frase "Il ragazzo desidera essere creduto dalla ragazza" produce lo stesso grafo: parole e sintassi diverse, significato standardizzato.

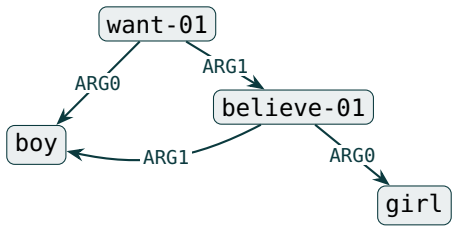


Figura 1.4. Il grafo AMR di "The boy wants the girl to believe him": lo stesso nodo `boy` è chi vuole (`ARG0` di `want-01`) ed è anche colui a cui la ragazza crede (`ARG1` di `believe-01`).

1.3 Il significato come vettore

Nel 2013 Mikolov et al. introducono **word2vec** [15], e cambia il modo di rappresentare il significato. Ogni parola diventa una lista di numeri, cioè un **vettore**, calcolato a partire dalle sue "compagnie" nei testi, ovvero dalle parole che le stanno accanto. Ne seguono due proprietà:

- **Parole simili finiscono vicine**: i sinonimi si raggruppano da soli, senza che nessuno l'abbia specificato.
- **Le direzioni hanno un senso**: la stessa direzione porta da *uomo* a *re* e da *donna* a *regina*, per cui re $-$ uomo $+$ donna $\approx$ regina (figura 1.5).

Da qui in poi, in NLP, il significato di una parola è un vettore, e tutto il resto è costruito sopra questa idea. Il dettaglio che conta è che il modello era piccolo e semplicissimo: la sorpresa fu che bastasse.

Dal vettore statico al vettore contestuale. In word2vec una parola ha **un solo vettore**, sempre lo stesso. Pensiamo alla parola *macchina*: il suo vettore sta a metà strada tra le auto, i dispositivi e la burocrazia, un compromesso che non descrive bene nessuno dei tre significati.

Con il Transformer (2017) [25], e poi con BERT e GPT (2018), il vettore di una parola viene invece **calcolato ogni volta**, guardando il contesto. La stessa parola finisce quindi in punti diversi dello spazio a seconda della frase: "Ho parcheggiato la macchina" la avvicina ad *auto*, "La macchina del caffè è rotta" a *elettrodomestico*, "La macchina amministrativa è lenta" a *burocrazia*.

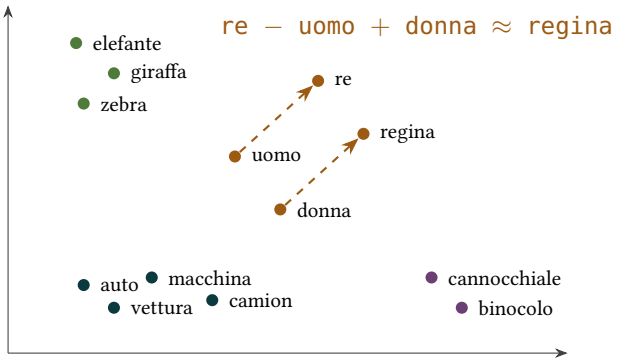


Figura 1.5. Uno spazio di word embedding disegnato in due dimensioni (nella realtà sono centinaia): parole simili finiscono vicine, e la stessa direzione porta da *uomo* a *re* e da *donna* a *regina*.

Polisemia e ambiguità si risolvono così "da sole", senza nessuna regola scritta: è il contesto a fare il lavoro, grazie a un meccanismo che si chiama **self-attention**.

1.4 Large Language Models

Un **modello di linguaggio** (language model) è un programma che, dato un testo, stima quanto è probabile ogni possibile parola successiva:

$$P(\text{parola} \mid \text{tutte le parole precedenti}).$$

Forniamo il seguente esempio: "Crostata con marmellata di …". Il modello assegna una probabilità a ogni parola del vocabolario (decine di migliaia di parole), e *albicocche* deve avere una probabilità alta, *salsiccia* bassa e *satellite* bassissima (figura 1.6). Per farlo bene il modello deve aver assorbito la grammatica (dopo "di" viene un nome), il lessico (le marmellate si fanno con la frutta) e il senso comune (di albicocche più che di kiwi).

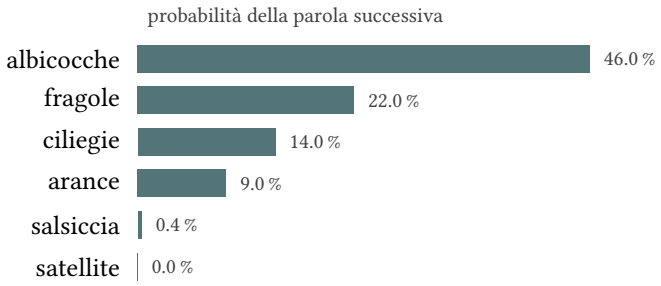


Figura 1.6. La distribuzione di probabilità sulla parola successiva a "Crostata con marmellata di …": *albicocche* è la più probabile, *satellite* ha probabilità praticamente nulla.

L'idea è vecchia, risale a Shannon (1948) [23], e la ritroveremo nel capitolo 3 con i modelli N-gram. La novità è realizzarla con reti neurali molto grandi e con vettori contestuali. Una precisazione: il modello non lavora su parole ma su **token**. Alcune parole possono essere

suddivise in più token (ad esempio “marmell”, “ata”), scelti in automatico in base alla frequenza, e il vocabolario è di 50–100 mila token[1].

Perché indovinare la parola insegna così tanto? Per predire bene la parola successiva bisogna avere sviluppato delle competenze precise:

- “La capitale della Francia è …”: i **fatti**;
- “Il trofeo non entra nella valigia perché è troppo grande. Il troppo grande è il …”: il **senso comune**;
- “Domanda: 17×3? Risposta: …”: il **calcolo**;
- “Inglese: house. Francese: …”: la **traduzione**;
- “Ho visto l’elefante con il cannocchiale, così ho potuto guardarlo da …”: la **disambiguazione**.

L’intuizione di GPT-2 (2019) è che un modello abbastanza grande, per predire meglio, finisce per imparare a svolgere i compiti che i testi “dimostrano”: riassumere, tradurre, rispondere, anche se nessuno glieli ha insegnati. Con GPT-3 (2020) basta poi mostrare due o tre esempi nella richiesta (*few-shot*) perché il modello continui lo schema: il compito si descrive a parole, e il linguaggio diventa l’interfaccia di programmazione.

Come si addestra un modello GPT. Si prende un testo, si nasconde la parola successiva e si chiede al modello la sua distribuzione di probabilità. Una funzione di **loss** misura l’errore, ovvero quanta poca probabilità è andata alla parola giusta. I **parametri** (i numeri interni del modello, i pesi delle connessioni) vengono poi ritoccati di pochissimo nella direzione che la riduce, con la discesa del gradiente che vedremo nel capitolo 5. Il processo continua con la parola successiva[2]. In questa fase non serve nessun annotatore umano, solo testo. La crescita dei modelli è stata rapida:

- **2017, Transformer**: l’architettura basata sulla self-attention [25].
- **2018, GPT-1**: 117 milioni di parametri, addestrato su 7000 libri (circa un miliardo di parole).
- **2019, GPT-2**: 1,5 miliardi di parametri e 40 GB di pagine web, svolge compiti mai insegnati.
- **2020, GPT-3**: 175 miliardi di parametri e 300 miliardi di parole, impara “al volo” da pochi esempi nel prompt.
- **2022, ChatGPT**: GPT-3.5 più un allineamento con feedback umano (InstructGPT), 100 milioni di utenti dopo due mesi.

Più parametri vogliono dire più capacità, a patto di avere abbastanza dati.

1.5 Espressioni regolari

Lasciamo la panoramica e passiamo al primo strumento concreto. Un’**espressione regolare** (regular expression, o regex) è un linguaggio formale per specificare stringhe di testo: in termini

[1] Vedremo come si costruisce un vocabolario di questo tipo nella sezione 2.3, con l’algoritmo BPE. Nel seguito diremo “parola” per semplicità, consapevoli che è un’approssimazione.

[2] In realtà è una semplificazione: la loss viene calcolata in parallelo su tutte le posizioni del testo.

più rigorosi, è una notazione algebrica che caratterizza un insieme di stringhe. Le regex si usano in ogni linguaggio di programmazione, in strumenti come `grep` e negli editor di testo, e servono sia a **cercare** un pattern in un testo sia a specificare come **modificarlo**. In Python, ad esempio, `re.search(pattern, string)` restituisce il primo match del pattern dentro la stringa.

Forniamo il seguente esempio, su cui torneremo: come cerchiamo in un testo tutte le occorrenze di *woodchuck*, *woodchucks*, *Woodchuck* e *Woodchucks*? Il pattern più semplice è una sequenza di caratteri, come `woodchuck`, che però trova solo la prima delle quattro forme: le regex distinguono maiuscole e minuscole. Servono quindi degli operatori.

1.5.1 Disgiunzioni

Le **parentesi quadre** specificano una disgiunzione di caratteri: il pattern fa match con uno qualsiasi dei caratteri tra parentesi. Con un trattino si specifica invece un **intervallo** (range) di caratteri. Alcuni esempi sono in tabella 1.1, dove è sottolineato il primo match.

Pattern	Match	Esempio
`[wW]oodchuck`	*Woodchuck* o *woodchuck*	Woodchuck
`[1234567890]`	una cifra qualsiasi	plenty of 7 to 5
`[A-Z]`	una lettera maiuscola	we should call it "Drenched Blossoms"
`[a-z]`	una lettera minuscola	my beans were impatient
`[0-9]`	una singola cifra	Chapter 1: Down the Rabbit Hole

Tabella 1.1. Disgiunzioni di caratteri con le parentesi quadre e con gli intervalli.

Il **caret** `^` subito dopo la parentesi quadra aperta nega la disgiunzione: `[^Ss]` fa match con qualsiasi carattere che non sia né *S* né *s*. Il caret ha questo significato solo se è il primo simbolo dentro le parentesi, altrimenti indica semplicemente il carattere `^` (tabella 1.2).

Pattern	Match	Esempio
`[^A-Z]`	non una lettera maiuscola	Oyfn pripetchik
`[^Ss]`	né *S* né *s*	I have no exquisite reason
`[^e^]`	né *e* né `^`	Look here
`a^b`	la stringa `a^b`	look up `a^b` now

Tabella 1.2. Il caret come negazione dentro le parentesi quadre, oppure come semplice carattere.

Le parentesi quadre non bastano però per disgiunzioni tra stringhe. *Woodchuck* è anche un altro nome della marmotta (*groundhog*), e per cercare l'una o l'altra parola si usa il **pipe**: `groundhog|woodchuck` fa match con entrambe, e `a|b|c` equivale a `[abc]`. I due operatori si combinano, ad esempio in `[gG]roundhog|[Ww]oodchuck`.

1.5.2 Contatori e caratteri jolly

Alcuni operatori dicono quante volte deve comparire il carattere che li precede:

- `?`: zero o una volta, quindi `colou?r` fa match con *color* e *colour*;

- `*` (**Kleene star**): zero o più volte, quindi `oo*h!` fa match con *oh!*, *ooh!*, *oooh!* e così via;
- `+` (**Kleene plus**): una o più volte, quindi `o+h!` fa match con le stesse stringhe di `oo*h!`, e `baa+` con *baa*, *baaa*, *baaaa*, …;
- `.`: un carattere qualsiasi (tranne il newline), quindi `beg.n` fa match con *begin*, *begun* e *beg3n*.

Il punto si usa spesso insieme alla Kleene star: `.*` indica una stringa qualsiasi, anche vuota, e lo ritroveremo in ELIZA. Esiste anche il contatore esplicito `{n}`, che indica esattamente n occorrenze: ad esempio `\d{2}` sono due cifre consecutive.

1.5.3 Backslash e ancore

Il **backslash** ha due usi. Il primo è riferirsi a un carattere speciale come carattere normale: `\.` è un punto, `\*` un asterisco, `\[` una parentesi quadra e `\\` il backslash stesso. Il secondo è indicare i caratteri di escape, come `\n` (newline) e `\t` (tab). Esistono inoltre degli **alias** per gli insiemi di caratteri più comuni (tabella 1.3).

Alias	Espansione	Match	Esempio
`\d`	`[0-9]`	una cifra	Party of 5
`\D`	`[^0-9]`	una non cifra	Blue moon
`\w`	`[a-zA-Z0-9_]`	alfanumerico o underscore	Daiyu
`\W`	`[^\w]`	non alfanumerico	!!!!
`\s`	`[ \r\t\n\f]`	spazio bianco	in Concord
`\S`	`[^\s]`	non spazio bianco	in Concord

Tabella 1.3. Alias per gli insiemi di caratteri più comuni.

Le **ancore** legano invece il pattern a una posizione precisa della stringa: `^` indica l'inizio della riga e `$` la fine. Ad esempio `^[A-Z]` fa match con la *P* di *Palo Alto* solo se si trova a inizio riga. Allo stesso modo `\.$` fa match con il punto finale di *The end.*, mentre `.$` con l'ultimo carattere della riga, che sia un punto, un punto interrogativo o un punto esclamativo.

Il caret ha quindi tre significati: inizio riga, negazione dentro le parentesi quadre, semplice carattere. Un'altra ancora utile è `\b`, il **confine di parola**: `\bthe\b` fa match con la parola *the* ma non con *other*[3].

1.5.4 Un esempio: trovare *the*

Vogliamo trovare in un testo tutte le occorrenze dell'articolo inglese *the*. Il pattern più semplice, `the`, non trova le occorrenze a inizio frase, scritte con la maiuscola. Possiamo correggerlo con `[tT]he`, che però restituisce anche il *the* contenuto in *other* o *theology*. Una prima soluzione è richiedere un carattere non alfabetico prima e dopo, con `[^a-zA-Z][tT]he[^a-zA-Z]`. Una seconda è usare i confini di parola, con `\b[tT]he\b`.

Le due soluzioni non sono equivalenti: la prima richiede che ci sia un carattere prima di *The*, e quindi non trova un *The* all'inizio della riga (figura 1.7).

[3] Per le regex una "parola" è una sequenza di lettere, cifre e underscore, come gli identificatori dei linguaggi di programmazione. Le ancore non consumano caratteri: fanno match con la stringa vuota in una certa posizione.

Il processo che abbiamo seguito si basa sulla correzione di due tipi di errore:

- **Falsi positivi**: stringhe trovate per errore, come *other* o *theology*.
- **Falsi negativi**: stringhe che avremmo dovuto trovare e che invece abbiamo perso, come *The*.

Ridurre l'errore complessivo richiede due sforzi tra loro antagonisti: aumentare la **precision** (ridurre i falsi positivi) e aumentare la **recall** (ridurre i falsi negativi). È un problema che torna di continuo in NLP, e daremo una definizione precisa delle due misure nella sezione 4.8.

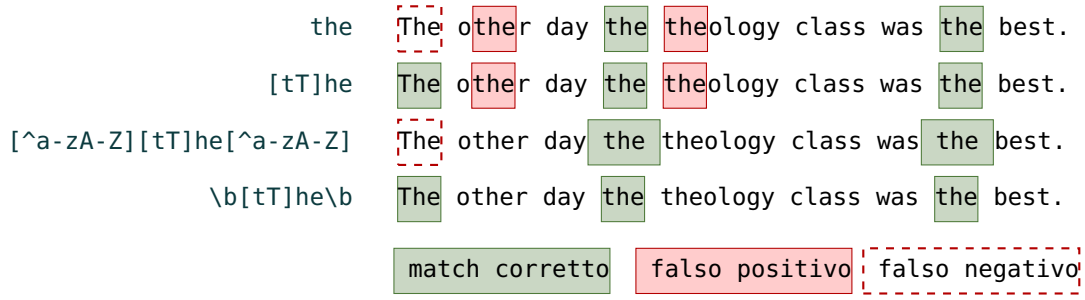


Figura 1.7. I quattro tentativi per trovare l'articolo *the* nella frase "The other day the theology class was the best.". Il pattern `the` perde il *The* iniziale e trova *the* dentro *other* e *theology*, mentre `[tT]he` recupera il *The* ma mantiene i falsi positivi. Il terzo pattern elimina i falsi positivi, ma senza un carattere prima del *The* iniziale lo perde di nuovo. Con i confini di parola `\b` si trovano tutte e sole le tre occorrenze.

1.5.5 Sostituzioni e capture group

Un uso molto comune delle regex è la **sostituzione**, in cui si sostituisce una stringa con un'altra. Nei comandi UNIX (e in molti editor) si scrive `s/regexp1/pattern/`: ad esempio `s/colour/color/` sostituisce *colour* con *color*. In Python si usa invece `re.sub(pattern, repl, string)`.

Spesso però la stringa da inserire dipende da quella trovata. Supponiamo di voler racchiudere tra parentesi angolari tutti i numeri di un testo, trasformando *the 35 boxes* in *the <35> boxes*. Lo strumento giusto è il **capture group**. Le parentesi tonde "catturano" la parte di testo che fa match e la memorizzano in un registro numerato (1, 2, 3, …, da sinistra verso destra). Nella sostituzione si fa poi riferimento al contenuto del registro con `\1`, `\2` e così via:

```
s/([0-9]+)/<\1>/
```

I registri si possono usare anche nel pattern stesso, per trovare ripetizioni. Il pattern

```
the (.*)er they (.*), the \1er we \2
```

fa match con *the faster they ran, the faster we ran*, ma non con *the faster they ran, the faster we ate*, perché nel secondo caso il contenuto del secondo registro (*ran*) non si ripete.

Le parentesi hanno quindi una doppia funzione: **raggruppano** i termini e **catturano** il match. Come raggruppamento servono a stabilire a cosa si applica un operatore[4]: `(The )?\w+` fa match con *The dog* e con *house*, mentre `(very )*challenging` fa match con *very very challenging*.

[4] Le sequenze di caratteri hanno precedenza sulla disgiunzione e i contatori hanno precedenza sulle sequenze. Per questo `guppy|ies` fa match con *guppy* o con *ies*, mentre per *guppy* o *guppies* bisogna scrivere `gupp(y|ies)`.

Quando vogliamo raggruppare senza catturare, usiamo un **non-capturing group**, scrivendo `?:` dopo la parentesi aperta. Ad esempio, `(?:some|a few) (people|cats) like some \1` fa match con *some cats like some cats* ma non con *some cats like some some*: il primo gruppo non viene memorizzato, quindi `\1` si riferisce al secondo.

1.5.6 Lookahead

A volte serve guardare avanti nel testo per controllare se un pattern fa match, senza però avanzare nella stringa. Le **lookahead assertion** fanno proprio questo:

- `(?= pattern)` è vera se *pattern* fa match, ma ha ampiezza zero (*zero-width*), ovvero non sposta il puntatore al carattere corrente;
- `(?! pattern)` è vera se *pattern* non fa match, anche questa con ampiezza zero.

Il lookahead negativo serve tipicamente a escludere un caso particolare. Ad esempio, per trovare a inizio riga una qualsiasi parola che non inizi con *Volcano* si scrive `^(?!Volcano)[A-Za-z]+`: il lookahead controlla che la riga non inizi con *Volcano*, poi `[A-Za-z]+` consuma la parola.

1.6 ELIZA

Una semplice applicazione delle regex è **ELIZA** [26], uno dei primi sistemi di NLP, realizzato da Joseph Weizenbaum nel 1966. ELIZA imita uno psicoterapeuta rogersiano[5] e usa il pattern matching per riconoscere frasi come "I need X", trasformandole in risposte come "What would it mean to you if you got X?". Una conversazione tipica è questa:

User:	Men are all alike.
ELIZA:	IN WHAT WAY
User:	They're always bugging us about something or other.
ELIZA:	CAN YOU THINK OF A SPECIFIC EXAMPLE
User:	Well, my boyfriend made me come here.
ELIZA:	YOUR BOYFRIEND MADE YOU COME HERE
User:	He says I'm depressed much of the time.
ELIZA:	I AM SORRY TO HEAR YOU ARE DEPRESSED

ELIZA funziona con una **cascata di sostituzioni**: ognuna fa match con una parte della frase in input e la trasforma. Le fasi sono due. Nella prima l'input viene portato in maiuscolo e si scambiano i pronomi, sostituendo *MY* con *YOUR*, *I'M* con *YOU ARE* e così via, in modo che quando ELIZA ripete una parte della frase sembri riferirsi correttamente all'utente. Nella seconda si applicano le regole che trasformano l'input in una risposta completa, come queste:

```
s/.* I'M (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/
s/.* I AM (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/
s/.* all .*/IN WHAT WAY?/
s/.* always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE?/
```

La figura 1.8 mostra la cascata completa usata nella versione web, dove si può conversare con ELIZA e vedere quale regola scatta a ogni frase.

[5] La terapia rogersiana è centrata sul paziente: il terapeuta riformula e restituisce al paziente ciò che dice. È un dominio in cui si può rispondere in modo credibile senza conoscere niente del mondo.



Figura 1.8. La cascata di regole di ELIZA. Dopo il passaggio in maiuscolo e lo scambio dei pronomi, la frase "Well, my boyfriend made me come here." diventa "WELL, YOUR BOYFRIEND MADE YOU COME HERE.": non contiene né *DEPRESSED*, né *ALL*, né *ALWAYS*, né *I NEED*, quindi scatta la regola 7, che ripete la frase a partire da *YOUR*.

ELIZA non capisce niente di quello che le viene detto, eppure molte delle persone che la usavano si convincevano del contrario. Al di là della curiosità storica, l'esempio mostra che le espressioni regolari hanno un ruolo sorprendentemente grande in NLP. Sequenze sofisticate di regex sono spesso il primo modello per qualsiasi elaborazione del testo. Per i compiti difficili si usano classificatori di machine learning, ma anche in quel caso le regex restano utili per il pre-processing e come feature dei classificatori, perché sono molto efficaci nel catturare generalizzazioni. Il prossimo capitolo comincia proprio da qui, con la tokenizzazione del testo.

Parte I

La pipeline classica

Capitolo 2

Normalizzazione del testo

Qualsiasi task di NLP richiede una **normalizzazione del testo**, che consiste in tre operazioni:

1. **tokenizzare** (segmentare) le parole;
2. **normalizzare** il formato delle parole;
3. **segmentare** le frasi.

Prima di vederle una alla volta, conviene però mettersi d'accordo su cosa sia una parola, dato che la risposta è meno ovvia di quanto sembri. Il riferimento per questo capitolo è il capitolo 2 di [11].

2.1 Parole e corpora

Quante parole ci sono in una frase? Consideriamo "Seuss's cat in the hat is different from other cats!". Le parole *cat* e *cats* sono la stessa parola oppure no? Dipende da cosa intendiamo:

- un **lemma** è un insieme di forme che hanno la stessa radice (stem), la stessa parte del discorso e, a grandi linee, lo stesso significato: *cat* e *cats* hanno lo stesso lemma;
- una **wordform** è invece la forma flessa completa, così come compare nel testo: *cat* e *cats* sono due wordform diverse.

Una seconda distinzione riguarda il conteggio. Un **word type** è un elemento del vocabolario, mentre un **word token** è una sua istanza nel testo[1]. La frase

they lay back on the San Francisco grass and looked at the stars and their

contiene 15 token e 13 type, dato che *the* e *and* compaiono due volte. Se però consideriamo *San Francisco* come un'unica parola, i token diventano 14 e i type 12, e con scelte ancora diverse (ad esempio raggruppando forme diverse della stessa parola) i type potrebbero scendere ancora. Anche una domanda così semplice, quindi, ha una risposta che dipende dal task.

Quante parole in un corpus? Indichiamo con N il numero di token e con V il **vocabolario**, ovvero l'insieme dei type, per cui $|V|$ è la dimensione del vocabolario (il numero di wordform distinte). La relazione tra le due quantità è descritta dalla **legge di Heaps** (o legge di Herdan):

$$|V| = kN^{\beta} \tag{2.1}$$

[1] Si parla anche di *word instance* al posto di word token, riservando il termine *token* all'output degli algoritmi di tokenizzazione in sottoparole che vedremo nella sezione 2.3.

dove k e β sono costanti positive, con $0 < \beta < 1$. I valori tipici di β vanno da circa 0.44 fino a 0.75, e dipendono dalla dimensione del corpus e dal genere. In ogni caso il vocabolario cresce un po' più velocemente della radice quadrata del numero di token (tabella 2.1).

Corpus	Token $= N$	Type $= \|V\|$
Conversazioni telefoniche Switchboard	2.4 milioni	20 mila
Shakespeare	884 mila	31 mila
COCA	440 milioni	2 milioni
Google N-grams	1000 miliardi	più di 13 milioni

Tabella 2.1. Numero di token e di type in alcuni corpora inglesi: più il corpus è grande, più type troviamo.

Il fatto che il vocabolario cresca senza fermarsi ha una conseguenza pratica: per quanto grande sia il vocabolario di un modello, prima o poi il modello incontrerà parole che non ha mai visto, le cosiddette **parole sconosciute**. È uno dei motivi per cui i modelli moderni non lavorano su parole intere ma su sottoparole, come vedremo nella sezione 2.3.

2.1.1 Corpora

Le parole non compaiono dal nulla. Un testo è prodotto da uno o più autori specifici, in un momento specifico, in una specifica varietà di una specifica lingua, e per una specifica funzione. I corpora variano quindi lungo diverse dimensioni:

- **Lingua**: nel mondo si parlano 7097 lingue, ed è opportuno testare gli algoritmi su più di una, mentre c'è la tendenza a svilupparli solo per l'inglese.
- **Varietà**: ad esempio le varietà dell'African American English, in cui un post su Twitter può contenere forme come *iont* (*I don't*), che influenzano la segmentazione in parole.
- **Code switching**: l'uso di più lingue nella stessa frase, come in "Por primera vez veo a @username actually being hateful! It was beautiful :)", in spagnolo e inglese.
- **Genere**: articoli di giornale, narrativa, articoli scientifici, Wikipedia.
- **Caratteristiche demografiche degli autori**: età, genere, etnia, condizione socio-economica.

Per questo motivo chi costruisce un corpus dovrebbe documentarlo con un **datasheet** [10] (o *data statement* [2]), che risponde a domande come: perché è stato raccolto il corpus, da chi e con quali finanziamenti (**motivazione**)? In quale situazione è stato scritto il testo (**situazione**)? Se è un sottoinsieme, come è stato campionato, c'era il consenso degli autori, che pre-processing è stato applicato (**processo di raccolta**)? A queste si aggiungono il processo di annotazione, la varietà linguistica e le caratteristiche demografiche di autori e annotatori.

2.2 Tokenizzazione delle parole

Il modo più semplice per tokenizzare un testo è la **tokenizzazione basata sugli spazi**: un token è tutto ciò che si trova tra due spazi. Funziona per le lingue che separano le parole con gli spazi,

ovvero quelle scritte con sistemi di scrittura basati sugli alfabeti arabo, cirillico, greco, latino e così via.

2.2.1 Tokenizzare con gli strumenti UNIX

Per l'inglese possiamo tokenizzare un testo e contare le frequenze delle parole con una sola riga di comando, usando `sed` (o `tr`) per spezzare il testo e `sort` e `uniq` per contare:

```
cat shakespeare.txt | sed -r 's/[^A-Za-z]+/\n/g' | sed '/^$/d' \
    | sort | uniq -c | sort -n -r
```

I comandi, collegati da una pipe, fanno nell'ordine (figura 2.1):

1. `sed -r 's/[^A-Za-z]+/\n/g'` trasforma ogni sequenza di caratteri non alfabetici in un a capo, così che ogni riga contenga una parola: sull'intero Shakespeare si ottiene *This*, *is*, *the*, *th*, *Etext*, *file*, *presented*, …;
2. `sed '/^$/d'` elimina le righe vuote;
3. `sort` ordina le righe in ordine alfabetico, per cui le parole uguali finiscono una sotto l'altra (all'inizio, una lunga serie di *A*);
4. `uniq -c` fonde le righe uguali adiacenti e conta quante sono, quindi ogni riga diventa un type con la sua frequenza;
5. `sort -n -r` ordina in ordine numerico decrescente.

Il risultato mette in cima le parole più frequenti, che in Shakespeare, come in qualsiasi corpus, sono le parole funzionali brevi: *the* (23288 volte), *I* (22225), *and* (18653), *to* (16373), *of* (15725), *a*, *you*, *my*, *in*. Lo stesso risultato si ottiene con `tr -sc 'A-Za-z' '\n'`, che sostituisce (`-c`, complemento) ogni carattere non alfabetico con un a capo e fonde (`-s`, squeeze) gli a capo consecutivi[2].

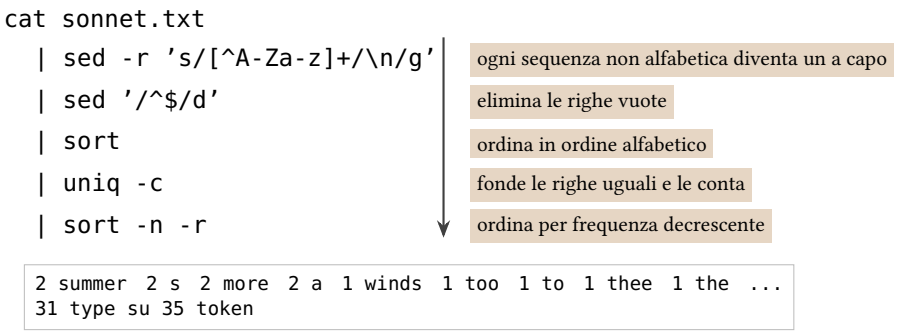


Figura 2.1. La pipeline UNIX per contare le parole di un testo, applicata ai primi quattro versi del sonetto 18 di Shakespeare ("Shall I compare thee to a summer's day? …"). Ogni comando riceve l'output del precedente, e in fondo restano i type ordinati per frequenza: *summer*, *s*, *more* e *a* compaiono due volte, tutti gli altri una.

[2] Aggiungendo `tr A-Z a-z` prima del primo `sort` si porta tutto in minuscolo, e *The* e *the* vengono contati insieme: è il case folding della sezione 2.4.1.

2.2.2 Problemi della tokenizzazione

Strumenti così semplici bastano per una statistica veloce, ma non possiamo limitarci a eliminare spazi e punteggiatura alla cieca. Ci sono infatti diversi casi da gestire:

- la punteggiatura interna alle parole, come in *m.p.h.*, *Ph.D.*, *AT&T* e *cap'n*;
- i prezzi (*$45.55*), che non vogliamo spezzare in *45* e *55*, e le date (*01/02/06*);
- gli URL (`http://www.stanford.edu`), gli hashtag (*#nlproc*) e gli indirizzi email (*someone@cs.colorado.edu*);
- i **clitici**, ovvero parti di parola che non possono stare da sole, come *are* in *we're*, il francese *je* in *j'ai* e *le* in *l'honneur*;
- le **multiword expression** (MWE), come *New York* o *rock 'n' roll*, per cui bisogna decidere se trattarle come un'unica parola.

Un tokenizzatore basato su regole decide come trattare ciascuno di questi casi. Uno degli standard più usati è quello del **Penn Treebank**, che separa i clitici (*doesn't* diventa *does* e *n't*), tiene insieme le parole con il trattino e separa tutta la punteggiatura. I modelli di linguaggio moderni, invece, usano spesso la tokenizzazione in sottoparole, in cui ad esempio *looking* può diventare *look* e *-ing*.

2.2.3 Tokenizzare con NLTK

Visto che la tokenizzazione viene eseguita prima di qualsiasi altra elaborazione, deve essere molto veloce. Per questo si usano algoritmi deterministici basati su espressioni regolari. La libreria NLTK [4], ad esempio, ha la funzione `regexp_tokenize`, che riceve il pattern dei token come espressione regolare:

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)     # set flag to strip out spaces and comments
...     (?:[A-Z]\.)+       # abbreviations, e.g. U.S.A.
...   | \w+(?:-\w+)*       # words with optional internal hyphens
...   | \$?\d+(?:\.\d+)?%? # currency and percentages, e.g. $12.40, 82%
...   | \.\.\.             # ellipsis
...   | [][.,;"'?():-_`]   # these are separate tokens; includes ], [
... '''
>>> nltk.regexp_tokenize(text, pattern, gaps=False)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Il flag `(?x)` permette di scrivere il pattern su più righe con i commenti. Ogni alternativa del pattern corrisponde a un tipo di token: le abbreviazioni, le parole con trattini interni, i prezzi e le percentuali, i puntini di sospensione e la punteggiatura.

Più in generale, in NLTK la funzione `word_tokenize()` usa il tokenizzatore migliore per la lingua indicata (di default l'inglese). I tokenizzatori sono organizzati in una gerarchia di classi che parte da `TokenizerI`, con i metodi `tokenize` e `span_tokenize`:

- **TreebankWordTokenizer**: tokenizza secondo lo standard del Penn Treebank;
- **RegexpTokenizer**: usa un'espressione regolare, come nell'esempio;
- **WordPunctTokenizer**: usa l'espressione regolare `\w+|[^\w\s]+`, quindi separa le sequenze alfanumeriche dalla punteggiatura;

- **WhitespaceTokenizer**: separa sugli spazi, le tabulazioni e gli a capo.

2.2.4 Lingue senza spazi

Molte lingue, come il cinese, il giapponese e il thai, non usano gli spazi per separare le parole. Come decidiamo allora dove finisce un token? In cinese le parole sono composte da caratteri chiamati **hanzi**, ognuno dei quali rappresenta un'unità di significato (un **morfema**, che definiremo nella sezione 2.3), e ogni parola ne contiene in media 2.4. Decidere cosa conti come parola, però, è complesso e non c'è un accordo. La frase "Yao Ming reaches the finals" della figura 2.2, ad esempio, può essere segmentata in tre parole, in cinque parole o in sette caratteri.



Figura 2.2. Tre segmentazioni possibili della frase cinese "Yao Ming reaches the finals": con lo standard del Chinese Treebank i nomi propri sono un'unica parola (3 parole), con lo standard della Peking University nome e cognome sono separati, così come alcuni aggettivi (5 parole), oppure si rinuncia alle parole e si usano i singoli caratteri (7 token).

In cinese è comune trattare semplicemente ogni carattere come un token, e la segmentazione diventa banale: i caratteri sono infatti a un livello semantico ragionevole per la maggior parte delle applicazioni. In altre lingue, come il thai e il giapponese, il singolo carattere è un'unità troppo piccola e serve una segmentazione più complessa. Gli algoritmi standard sono in questo caso modelli neurali per sequenze, addestrati in modo supervisionato.

2.3 Tokenizzazione in sottoparole

Invece di segmentare sugli spazi o sui singoli caratteri, possiamo lasciare che siano i dati a dirci come tokenizzare. Si parla di **tokenizzazione in sottoparole** (subword tokenization), perché i token possono essere sia parole intere sia parti di parole. Il vantaggio principale riguarda le parole sconosciute: se nel training abbiamo visto *low*, *new* e *newer* ma non *lower*, quando *lower* compare nel test possiamo comunque rappresentarla con due sottoparole note, *low* ed *er*. Nel caso peggiore, una parola molto rara può essere rappresentata come una sequenza di singoli caratteri.

Gli algoritmi più diffusi sono tre: il **Byte-Pair Encoding** (BPE) [22], la tokenizzazione basata su un **unigram language model** [14] e **WordPiece** [21]. Tutti hanno due parti:

- un **token learner**, che riceve un corpus di training e ne induce un vocabolario, ovvero un insieme di token distinti;
- un **token segmenter**, che riceve una frase di test e la tokenizza secondo quel vocabolario.

2.3.1 Il token learner di BPE

Il token learner di BPE parte da un vocabolario che contiene tutti i singoli caratteri del corpus, $\{A, B, C, D, \ldots, a, b, c, d, \ldots\}$, e ripete i passi seguenti:

1. sceglie i due simboli che compaiono più spesso uno accanto all'altro nel corpus, ad esempio `A` e `B`;
2. aggiunge al vocabolario il nuovo simbolo `AB`;
3. sostituisce nel corpus ogni coppia adiacente `A B` con `AB`.

L'algoritmo si ferma dopo k merge, per cui k è un suo parametro, e il vocabolario finale contiene i caratteri iniziali più k nuovi simboli (algoritmo 1).

Algoritmo 1 Il token learner di BPE.

function BYTEPAIRENCODING(stringhe C, numero di merge k)
  $V \leftarrow$ tutti i caratteri distinti di C ▷ i token iniziali sono i caratteri
  for $i = 1$ to k **do** ▷ k merge
    $t_L, t_R \leftarrow$ coppia di token adiacenti più frequente in C
    $t_{NEW} \leftarrow t_L + t_R$ ▷ il nuovo token è la concatenazione
    $V \leftarrow V + t_{NEW}$ ▷ aggiorna il vocabolario
    sostituisci ogni occorrenza di t_L, t_R in C con t_{NEW} ▷ aggiorna il corpus
  end for
  return V
end function

In pratica, BPE viene eseguito solo dentro le parole: i merge non attraversano mai il confine tra due parole. Per questo si aggiunge prima un simbolo speciale di fine parola _ prima di ogni spazio, e poi si separa ogni parola nei suoi caratteri[3]. Forniamo il seguente esempio, che seguiremo per tutta la sezione. Il corpus di training è:

low low low low low lowest lowest newer newer newer newer newer newer wider wider wider new new

Aggiungendo i simboli di fine parola e contando le parole uguali, la sua rappresentazione e il vocabolario iniziale sono:

corpus	vocabolario
5 l o w _	_, d, e, i, l, n, o, r, s, t, w
2 l o w e s t _	
6 n e w e r _	
3 w i d e r _	
2 n e w _	

La coppia adiacente più frequente è `e r`, che compare in *newer* (6 volte) e in *wider* (3 volte), quindi 9 volte in tutto[4]. La uniamo in `er`, la aggiungiamo al vocabolario e la sostituiamo nel corpus, che diventa `n e w er _` e `w i d er _`. A questo punto la coppia più frequente è `er _` (9 volte), che diventa `er_`, poi `n e` (8 volte), che diventa `ne`. I merge successivi sono:

[3] Nei modelli di linguaggio si usa spesso una convenzione diversa: lo spazio viene attaccato all'inizio della parola successiva. Il testo viene prima pre-tokenizzato con un'espressione regolare, separandolo su spazi e punteggiatura, e i merge sono permessi solo dentro i pezzi così ottenuti.

[4] Anche `r _` compare 9 volte: a parità di frequenza si sceglie la coppia incontrata per prima scorrendo il corpus.

Merge	Token aggiunti al vocabolario (oltre ai caratteri iniziali)
(ne, w)	er, er_, ne, new
(l, o)	er, er_, ne, new, lo
(lo, w)	er, er_, ne, new, lo, low
(new, er_)	er, er_, ne, new, lo, low, newer_
(low, _)	er, er_, ne, new, lo, low, newer_, low_

La figura 2.3 riassume gli otto merge e lo stato finale del corpus.

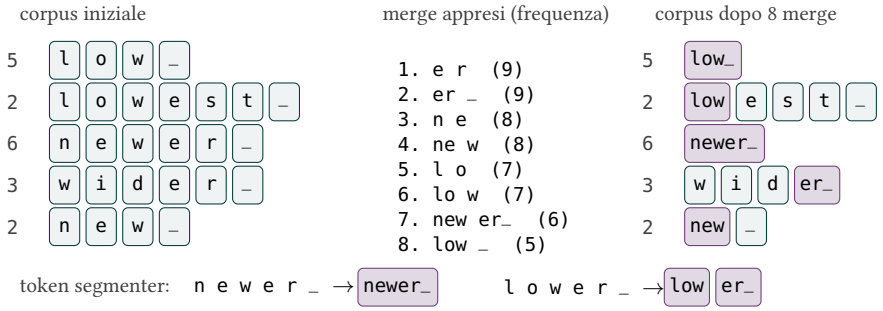


Figura 2.3. Gli otto merge appresi da BPE sul corpus dell'esempio: a sinistra il corpus di partenza, al centro i merge in ordine con la loro frequenza, a destra il corpus finale, con i token nuovi evidenziati. In basso, il token segmenter applica gli stessi merge a due parole di test: *newer* è un token unico, mentre *lower*, mai vista nel training, diventa `low` ed `er_`.

2.3.2 Il token segmenter di BPE

Sui dati di test il segmenter applica i merge appresi dal training, **nell'ordine in cui sono stati appresi**, mentre le frequenze nei dati di test non hanno alcun ruolo. Torniamo al nostro esempio: prima si sostituisce ogni `e r` con `er`, poi ogni `er _` con `er_`, e così via. La parola di test `n e w e r _` diventa quindi un unico token, `newer_`, mentre `l o w e r _` diventa due token, `low` ed `er_`. Nei casi reali BPE viene eseguito con decine di migliaia di merge su corpora molto grandi, e produce vocabolari di 50, 100 o anche 200 mila token: la maggior parte delle parole è rappresentata da un unico token, e solo le parole rare (e quelle sconosciute) vengono spezzate in più token.

2.3.3 Proprietà dei token BPE

I token prodotti da BPE contengono di solito le parole frequenti e le sottoparole frequenti, che spesso sono **morfemi** come *-est* o *-er*. Un morfema è la più piccola unità dotata di significato di una lingua: ad esempio *unlikeliest* ha tre morfemi, *un-*, *likely* e *-est*. Nel nostro esempio, BPE ha imparato da solo che *er_* è un pezzo ricorrente a fine parola, senza che nessuno gli abbia spiegato cosa sia un comparativo.

2.4 Normalizzazione delle parole

La **normalizzazione delle parole** consiste nel portare parole e token in un formato standard. Ad esempio *U.S.A.* e *USA* dovrebbero essere la stessa parola, così come *uhhuh* e *uh-huh*. Con scelte

più aggressive possiamo ricondurre alla stessa forma anche *am*, *is*, *be* e *are*, oppure, in italiano, *vado*, *andate* e *andare*.

2.4.1 Case folding

Il **case folding** riduce tutte le lettere al minuscolo. In applicazioni come l'information retrieval è la scelta più comune, visto che gli utenti tendono a scrivere in minuscolo. Una possibile eccezione sono le maiuscole a metà frase, come in *General Motors*, *Fed* (la Federal Reserve) contro *fed* e *SAIL* contro *sail*. Per altri task, come la sentiment analysis, la traduzione automatica e l'information extraction, le maiuscole sono invece un'informazione utile: *US* (gli Stati Uniti) e *us* (il pronome) sono parole diverse.

2.4.2 Lemmatizzazione

La **lemmatizzazione** rappresenta ogni parola con il suo **lemma**, ovvero la radice comune, che coincide con la forma che troveremmo in un dizionario:

- *am*, *are*, *is* $\rightarrow$ *be*;
- *car*, *cars*, *car's*, *cars'* $\rightarrow$ *car*;
- lo spagnolo *quiero* ("voglio") e *quieres* ("vuoi") $\rightarrow$ *querer* ("volere");
- *He is reading detective stories* $\rightarrow$ *He be read detective story*.

La lemmatizzazione si fa con il **parsing morfologico**. Le parole sono fatte di **morfemi**, che si dividono in due classi: gli **stem**, che portano il significato principale, e gli **affissi**, che si attaccano agli stem e hanno spesso una funzione grammaticale. Un parser morfologico scompone *cats* nei due morfemi *cat* e *s*, e l'italiano *ameranno* nel morfema *amare* più le feature morfologiche `3PL` (terza persona plurale) e futuro.

2.4.3 Stemming

Lo **stemming** è una versione più grezza della lemmatizzazione: riduce i termini ai loro stem tagliando gli affissi in modo approssimativo, senza alcuna analisi. Ad esempio il testo

> *This was not the map we found in Billy Bones's chest, but an accurate copy, complete in all things-names and heights and soundings-with the single exception of the red crosses and the written notes.*

diventa:

> *Thi wa not the map we found in Billi Bone s chest but an accur copi complet in all thing name and height and sound with the singl except of the red cross and the written note*

Il risultato non è leggibile, ma forme diverse della stessa parola finiscono sulla stessa forma: *crosses* diventa *cross* e *notes* diventa *note*.

Lo stemmer più noto è il **Porter stemmer** [19], basato su una serie di regole di riscrittura eseguite in cascata: l'output di ogni passo diventa l'input del successivo. Alcune regole di esempio sono:

ATIONAL	→	ATE	(*relational* → *relate*)
ING	→	ϵ	se lo stem contiene una vocale (*motoring* → *motor*)
SSES	→	SS	(*grasses* → *grass*)

Per molte lingue, però, gestire una morfologia complessa è necessario e regole così semplici non bastano. Pensiamo alla parola turca *Uygarlastiramadiklarimizdanmissinizcasina*, che significa "(comportandovi) come se foste tra quelli che non siamo riusciti a civilizzare". È composta da undici morfemi: *Uygar* (civilizzato), *las* (diventare), *tir* (causare), *ama* (non essere in grado), *dik* (passato), *lar* (plurale), *imiz* (prima persona plurale), *dan* (ablativo), *mis* (passato), *siniz* (seconda persona plurale), *casina* (come se).

2.4.4 Lemmatizzare con spaCy

In pratica, la lemmatizzazione si fa con librerie come **spaCy**, le cui pipeline per l'inglese includono un lemmatizzatore basato su regole:

```python
import spacy

# English pipelines include a rule-based lemmatizer
nlp = spacy.load("en_core_web_sm")
lemmatizer = nlp.get_pipe("lemmatizer")
print(lemmatizer.mode)  # 'rule'

doc = nlp("I was reading the paper.")
print([token.lemma_ for token in doc])
# ['I', 'be', 'read', 'the', 'paper', '.']
```

Ogni token del documento ha l'attributo `lemma_` con il suo lemma, e allo stesso modo spaCy fornisce le feature morfologiche di ogni token[5].

2.5 Segmentazione delle frasi

L'ultima operazione della normalizzazione è la **segmentazione delle frasi**. Il punto esclamativo e il punto interrogativo sono segnali di fine frase quasi sempre non ambigui. Il punto, invece, è molto ambiguo, perché può indicare:

- la fine di una frase;
- un'abbreviazione, come *Inc.* o *Dr.*;
- un numero, come *.02%* o *4.3*.

I casi possono anche sovrapporsi: in "…he works for Apple Inc." il punto finale chiude sia l'abbreviazione sia la frase. Per questo la tokenizzazione delle parole e quella delle frasi vengono spesso affrontate insieme. L'algoritmo più comune è quindi tokenizzare prima, e poi usare delle regole o un modello di machine learning per classificare ogni punto come parte della parola oppure come confine di frase. Un dizionario delle abbreviazioni aiuta, e a quel punto la segmentazione in frasi si può fare spesso con semplici regole basate sulla tokenizzazione.

[5] La documentazione di spaCy su lemmatizzazione e morfologia è in `https://spacy.io/usage/linguistic-features`.

In NLTK la segmentazione si fa con la funzione `nltk.sent_tokenize()`, oppure con la classe `PunktSentenceTokenizer`, che può essere addestrata su un corpus specifico, ad esempio il file `overheard.txt` del corpus `webtext`:

```
from nltk.corpus import webtext
from nltk.tokenize import PunktSentenceTokenizer

t = PunktSentenceTokenizer(webtext.raw('overheard.txt'))
t.tokenize('First sentence. Second sentence')
```

Capitolo 3

Modelli di linguaggio N-gram

L'obiettivo di questo capitolo è **assegnare una probabilità a una frase**. A cosa serve? Ad esempio:

- nella **traduzione automatica**, $P(\text{high winds tonight}) > P(\text{large winds tonight})$ ci dice quale delle due traduzioni è più naturale;
- nella **correzione ortografica**, in "The office is about fifteen minuets from my house" il modello dovrebbe accorgersi che

$$P(\text{about fifteen minutes from}) > P(\text{about fifteen minuets from});$$

- nel **riconoscimento del parlato**, $P(\text{I saw a van}) \gg P(\text{eyes awe of an})$, anche se le due sequenze suonano quasi uguali;

e lo stesso vale per il riassunto automatico, il question answering e molti altri task. Il motivo più importante, però, è un altro: i Large Language Models (sezione 1.4) sono addestrati proprio a predire la parola successiva. In questo capitolo vediamo il modello di linguaggio più semplice, il modello **N-gram**. È un modello che si formalizza in modo molto chiaro, e per questo ci permette di introdurre concetti che ritroveremo anche con i modelli neurali: training e test set, perplexity, campionamento e smoothing. Il riferimento per questo capitolo è il capitolo 3 di [11], insieme all'appendice C per il Kneser-Ney smoothing.

3.1 Modelli di linguaggio probabilistici

Ricordiamo la definizione di probabilità condizionata, che possiamo riscrivere come:

$$P(B \mid A) = \frac{P(A,B)}{P(A)} \qquad \Longrightarrow \qquad P(A,B) = P(A)\,P(B \mid A)$$

dove scriviamo $P(A,B)$ al posto di $P(A \cap B)$. Con più variabili si ottiene $P(A,B,C,D) = P(A)\,P(B \mid A)\,P(C \mid A,B)\,P(D \mid A,B,C)$, e in generale la **regola della catena** (chain rule):

$$P(X_{1:n}) = P(X_1)\,P(X_2 \mid X_1) \cdots P(X_n \mid X_{1:n-1}) = \prod_{i=1}^{n} P(X_i \mid X_{1:i-1}) \tag{3.1}$$

dove $X_{1:n}$ indica la sequenza $X_1 X_2 \ldots X_n$.

L'obiettivo è calcolare la probabilità di una sequenza di parole, $P(W_{1:n}) = P(W_1 W_2 \dots W_n)$. Un task collegato è calcolare la probabilità della parola successiva, $P(W_n \mid W_{1:n-1})$. Un modello che calcola una di queste due quantità si chiama **modello di linguaggio** (language model, LM).

I due task sono equivalenti. Se sappiamo calcolare $P(W_{1:n})$, otteniamo la probabilità della parola successiva dalla definizione di probabilità condizionata:

$$P(W_n \mid W_{1:n-1}) = \frac{P(W_{1:n-1}, W_n)}{P(W_{1:n-1})} = \frac{P(W_{1:n})}{P(W_{1:n-1})}.$$

Viceversa, se sappiamo calcolare $P(W_n \mid W_{1:n-1})$, otteniamo $P(W_{1:n})$ con la regola della catena:

$$P(W_{1:n}) = \prod_{i=1}^{n} P(W_i \mid W_{1:i-1}). \tag{3.2}$$

Ad esempio:

$$\begin{aligned} P(\text{its water is so transparent}) = {} & P(\text{its})\, P(\text{water} \mid \text{its})\, P(\text{is} \mid \text{its water}) \\ & \cdot P(\text{so} \mid \text{its water is})\, P(\text{transparent} \mid \text{its water is so}). \end{aligned}$$

Basta contare? Come stimiamo queste probabilità? Si potrebbe pensare di contare e dividere:

$$P(\text{the} \mid \text{its water is so transparent that}) = \frac{C(\text{its water is so transparent that the})}{C(\text{its water is so transparent that})}$$

dove $C(\cdot)$ è il numero di volte che una sequenza compare in un corpus. L'idea non è sbagliata, ma non funziona: le frasi possibili sono troppe, il linguaggio è creativo e frasi nuove vengono inventate di continuo. Nemmeno l'intero web basterebbe ad avere stime affidabili per sequenze così lunghe.

3.2 L'assunzione di Markov

L'idea dei modelli N-gram è approssimare la storia con le sole ultime parole. Con l'**assunzione di Markov** scriviamo ad esempio:

$$P(\text{the} \mid \text{its water is so transparent that}) \approx P(\text{the} \mid \text{that})$$

oppure, guardando due parole indietro, $P(\text{the} \mid \text{transparent that})$. In altre parole, approssimiamo ogni fattore del prodotto (3.2) guardando solo le $k-1$ parole precedenti[1]:

$$P(W_i \mid W_{1:i-1}) \approx P(W_i \mid W_{i-k+1:i-1}) \qquad \Longrightarrow \qquad P(W_{1:n}) \approx \prod_{i=1}^{n} P(W_i \mid W_{i-k+1:i-1}). \tag{3.3}$$

Il caso più semplice è il modello **unigram**, in cui ogni parola è indipendente dalle altre: $P(W_{1:n}) \approx \prod_i P(W_i)$. Le frasi generate da un modello unigram sono sequenze di parole frequenti

[1] Con $k = 2$ si parla di bigrammi, con $k = 3$ di trigrammi, e in generale di N-gram. Il termine N-gram indica sia la sequenza di N parole sia il modello probabilistico che la usa.

senza alcun legame tra loro, come “fifth an of futures the an incorporated a a the inflation most dollars quarter in is mass”.

Il modello **bigram** condiziona invece sulla parola precedente, $P(W_i \mid W_{1:i-1}) \approx P(W_i \mid W_{i-1})$, e le frasi generate iniziano ad avere una coerenza locale: “texaco rose one in this issue is pursuing growth in a boiler house said mr. gurria mexico’s motion control proposal without permission from five hundred fifty five yen”.

Possiamo estendere il modello a trigrammi, 4-grammi e 5-grammi. In generale un modello N-gram non è un modello sufficiente del linguaggio, perché il linguaggio ha dipendenze a lunga distanza. In “The computer which I had just put into the machine room on the fifth floor crashed”, ad esempio, *crashed* dipende da *computer*, che si trova quindici parole prima. Nella pratica, però, i modelli N-gram funzionano spesso abbastanza bene.

3.3 Stimare le probabilità

Il modo più intuitivo per stimare le probabilità di un modello N-gram è la **stima di massima verosimiglianza** (maximum likelihood estimate, MLE). La MLE dei parametri di un modello M su un training set T è quella che massimizza la verosimiglianza di T dato M, ovvero $P(T \mid M)$.

Forniamo il seguente esempio: la parola *bagel* compare 400 volte in un corpus di un milione di parole. Qual è la probabilità che una parola presa a caso da un altro testo sia *bagel*? La MLE è $400/1\,000\,000 = 0.0004$. Potrebbe essere una pessima stima per un altro corpus, ma è la stima che rende più probabile che *bagel* compaia 400 volte in un corpus di un milione di parole.

Per i bigrammi, la MLE si ottiene contando le occorrenze del bigramma e normalizzando:

$$P(w_i \mid w_{i-1}) = \frac{C(w_{i-1}w_i)}{\sum_w C(w_{i-1}w)} = \frac{C(w_{i-1}w_i)}{C(w_{i-1})} \tag{3.4}$$

dove il secondo passaggio vale perché la somma dei conteggi di tutti i bigrammi che iniziano con w_{i-1} è proprio il conteggio dell’unigramma w_{i-1}. Il rapporto tra la frequenza di una sequenza e quella del suo prefisso si chiama **frequenza relativa**.

Un esempio. Consideriamo un mini corpus di tre frasi, a cui aggiungiamo il simbolo `<s>` all’inizio, per avere il contesto della prima parola, e il simbolo `</s>` alla fine (vedremo tra poco a cosa serve):

```
<s> I am Sam </s>    <s> Sam I am </s>    <s> I do not like green eggs
                                          and ham </s>
```

Alcune probabilità stimate con l’equazione (3.4) sono:

$$P(\text{I} \mid \texttt{<s>}) = \tfrac{2}{3} = 0.67 \qquad P(\text{Sam} \mid \texttt{<s>}) = \tfrac{1}{3} = 0.33 \qquad P(\text{am} \mid \text{I}) = \tfrac{2}{3} = 0.67$$
$$P(\texttt{</s>} \mid \text{Sam}) = \tfrac{1}{2} = 0.5 \qquad P(\text{Sam} \mid \text{am}) = \tfrac{1}{2} = 0.5 \qquad P(\text{do} \mid \text{I}) = \tfrac{1}{3} = 0.33$$

Queste probabilità sono i **parametri** del modello, ed è proprio quello che memorizziamo.

> **Osservazione (Perché serve `</s>`).** Ogni probabilità condizionata del modello somma a 1 sulle parole che possono seguire il contesto. Senza `</s>`, quindi, il modello non ha modo di

fermarsi: fissata una lunghezza n, le probabilità di tutte le frasi di n parole sommano a 1, e questo vale per ogni n. Sommando su tutte le lunghezze si ottiene $1 + 1 + 1 + \ldots$, per cui il modello non è una distribuzione di probabilità sulle frasi, ma una distribuzione diversa per ogni lunghezza.
Consideriamo ad esempio un vocabolario di due sole parole, *a* e *b*, ognuna con probabilità $1/2$ dopo qualsiasi contesto. Le frasi di una parola (*a* e *b*) sommano a $1/2 + 1/2 = 1$, e quelle di due parole (*aa*, *ab*, *ba* e *bb*) sommano di nuovo a $4 \cdot 1/4 = 1$.
Aggiungiamo ora `</s>`: dopo ogni parola il modello sceglie tra *a*, *b* e `</s>`, ad esempio con probabilità $1/3$ ciascuna, mentre dopo `<s>` le due parole restano a $1/2$, perché una frase non può essere vuota. Una frase di n parole ha probabilità $\frac{1}{2} \cdot \left(\frac{1}{3}\right)^n$ (la prima parola, le $n-1$ successive e la fine della frase), e le frasi di n parole sono 2^n, quindi:

$$\sum_{n=1}^{\infty} 2^n \cdot \frac{1}{2} \left(\frac{1}{3}\right)^n = \frac{1}{2} \sum_{n=1}^{\infty} \left(\frac{2}{3}\right)^n = \frac{1}{2} \cdot 2 = 1.$$

La probabilità di fermarsi rende le frasi lunghe meno probabili, ed è grazie a lei che le probabilità di tutte le frasi, di qualsiasi lunghezza, sommano a 1.

3.3.1 Il Berkeley Restaurant Project

Passiamo a un corpus reale, anche se piccolo: le frasi del *Berkeley Restaurant Project* (BeRP), un sistema di dialogo che rispondeva a domande su un database di ristoranti di Berkeley. Le frasi sono richieste come "can you tell me about any good cantonese restaurants close by", "tell me about chez panisse" o "i'm looking for a good place to eat breakfast". La tabella 3.1 riporta i conteggi di alcuni bigrammi, calcolati su 9222 frasi[2].

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0
unigram	2533	927	2417	746	158	1093	341	278

Tabella 3.1. Conteggi dei bigrammi per otto parole del Berkeley Restaurant Project: la cella nella riga *i* e nella colonna *want* indica che *want* segue *i* 827 volte. In fondo, i conteggi degli unigrammi. La maggior parte dei valori è zero.

Dividendo ogni riga per il conteggio dell'unigramma corrispondente si ottengono le probabilità della tabella 3.2.

[2] Altre versioni del corpus contano 9332 frasi. Il corpus è disponibile in `https://github.com/wooters/berp-trans`.

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Tabella 3.2. Probabilità dei bigrammi della tabella 3.1, ottenute dividendo ogni riga per il conteggio dell'unigramma.

Con qualche altra probabilità, come $P(\text{i} \mid \texttt{<s>}) = 0.25$, $P(\text{english} \mid \text{want}) = 0.0011$, $P(\text{food} \mid \text{english}) = 0.5$ e $P(\texttt{</s>} \mid \text{food}) = 0.68$, possiamo calcolare la probabilità di una frase moltiplicando le probabilità dei suoi bigrammi:

$$\begin{aligned} P(\texttt{<s>}\ \text{i want english food}\ \texttt{</s>}) &= P(\text{i} \mid \texttt{<s>})\, P(\text{want} \mid \text{i})\, P(\text{english} \mid \text{want}) \\ &\quad \cdot P(\text{food} \mid \text{english})\, P(\texttt{</s>} \mid \text{food}) \\ &= 0.25 \times 0.33 \times 0.0011 \times 0.5 \times 0.68 = 0.000031. \end{aligned}$$

Che tipo di conoscenza c'è nei bigrammi? Alcune probabilità codificano fatti sintattici: $P(\text{to} \mid \text{want}) = 0.66$ dice che dopo *want* viene spesso un infinito, $P(\text{eat} \mid \text{to}) = 0.28$ che dopo *to* viene spesso un verbo, mentre $P(\text{food} \mid \text{to}) = 0$ e $P(\text{want} \mid \text{spend}) = 0$. Altre riguardano il task: $P(\text{i} \mid \texttt{<s>}) = 0.25$ riflette il fatto che le richieste a un assistente iniziano spesso con *I*. Altre ancora sono culturali più che linguistiche: $P(\text{chinese} \mid \text{want}) = 0.0065$ è molto più alta di $P(\text{english} \mid \text{want}) = 0.0011$.

3.3.2 Problemi pratici

Le probabilità sono numeri minori di 1, e più ne moltiplichiamo più il prodotto diventa piccolo, fino all'underflow. Per questo i calcoli si fanno sempre nello spazio dei logaritmi:

$$\log(p_1 \cdot p_2 \cdot p_3) = \log p_1 + \log p_2 + \log p_3$$

e in più sommare è più veloce che moltiplicare. Per costruire modelli N-gram reali si usano toolkit come SRILM e KenLM (utilizzabile anche in Python). Nel 2006 Google ha rilasciato un corpus di N-gram calcolato su mille miliardi di parole del web. Al suo interno, ad esempio, *serve as the incubator* compare 99 volte, *serve as the independent* 794 volte e *serve as the index* 223 volte.

Esercizio 3.1 (KenLM sul Berkeley Restaurant Project). Calcolare la probabilità e la perplexity (sezione 3.4.1) di un testo a piacere, usando un modello a bigrammi addestrato sul Berkeley Restaurant Project con il toolkit KenLM (`https://github.com/kpu/kenlm`).

3.4 Valutare un modello di linguaggio

Il nostro modello preferisce le frasi buone a quelle cattive? In altre parole, assegna una probabilità più alta alle frasi "reali" o osservate spesso, e più bassa a quelle sgrammaticate o rare? Per rispondere servono tre ingredienti:

- un **training set**, su cui stimiamo i parametri del modello;
- un **test set**, un insieme di dati mai visto, diverso dal training set e mai usato in fase di training, su cui misuriamo le prestazioni;
- una **metrica di valutazione**, che ci dice quanto è buono il modello sul test set.

Il test set non deve finire nel training set: se una frase di test è anche nel corpus di training, il modello le assegna una probabilità artificialmente alta (si parla di *data contamination*). Per lo stesso motivo il test set si usa una sola volta, alla fine, e tutte le prove si fanno su un terzo insieme, il **development set** (o devset).

Valutazione estrinseca e intrinseca. Il modo migliore per confrontare due modelli A e B è la **valutazione estrinseca** (o *in vivo*): si inserisce ciascun modello in un task, come un correttore ortografico, un riconoscitore del parlato o un sistema di traduzione. Si esegue poi il task e si confrontano le accuratezze ottenute (quante parole errate sono state corrette, quante parole sono state tradotte correttamente). Il problema è che la valutazione estrinseca richiede molto tempo, anche giorni o settimane. Per questo si usa spesso una **valutazione intrinseca**, che misura la qualità del modello indipendentemente da qualsiasi applicazione: la **perplexity**.

3.4.1 Perplexity

L'intuizione è quella dello **Shannon game**: quanto bene riusciamo a predire la parola successiva? Pensiamo a frasi come "I always order pizza with cheese and …", "The 33rd President of the US was …" o "I saw a …". Un buon modello, dopo la prima, assegna probabilità alte a *mushrooms* (0.1) e *pepperoni* (0.1), più basse ad *anchovies* (0.01) e bassissime a *fried rice* (0.0001) o ad *and* (10^{-100}).

Gli unigrammi sono pessimi in questo gioco, perché ignorano completamente il contesto. Un modello è quindi migliore se assegna una probabilità più alta alla parola che compare davvero.

La **perplexity** è l'inverso della probabilità del test set, normalizzato per il numero di parole:

$$\mathrm{PP}(W) = P(W_{1:n})^{-\frac{1}{n}} = \sqrt[n]{\frac{1}{P(W_{1:n})}}. \tag{3.5}$$

Con la regola della catena, e poi con l'approssimazione a bigrammi, diventa:

$$\mathrm{PP}(W) = \sqrt[n]{\prod_{i=1}^{n} \frac{1}{P(W_i \mid W_{1:i-1})}} \qquad \text{e} \qquad \mathrm{PP}(W) = \sqrt[n]{\prod_{i=1}^{n} \frac{1}{P(W_i \mid W_{i-1})}}. \tag{3.6}$$

Per via dell'inverso, **minimizzare la perplexity equivale a massimizzare la probabilità** del test set: il modello migliore è quello che predice meglio un testo mai visto, ovvero che gli assegna la probabilità più alta. La normalizzazione per n serve invece a rendere la misura indipendente

dalla lunghezza del testo, visto che la probabilità di un testo diminuisce man mano che il testo si allunga.

Perplexity come fattore di ramificazione. La perplexity si può interpretare come il **fattore di ramificazione medio pesato**, ovvero il numero medio di parole tra cui il modello deve scegliere a ogni passo. Supponiamo che una frase sia composta da n cifre casuali, e che il modello assegni probabilità $P(W_i) = 1/10$ a ogni cifra. La perplexity della frase è:

$$\mathrm{PP}(W_{1:n}) = P(W_{1:n})^{-\frac{1}{n}} = \left(\left(\frac{1}{10}\right)^n\right)^{-\frac{1}{n}} = \left(\frac{1}{10}\right)^{-1} = 10$$

ovvero il modello è "perplesso" come se a ogni passo dovesse scegliere tra 10 alternative equiprobabili.

Un esempio meno banale è quello di Josh Goodman. Un sistema di smistamento delle chiamate riceve 120 mila chiamate e deve riconoscere *R&D*, *Sales* e *Technical Support*, ognuna in una chiamata su quattro. Nella chiamata restante deve invece riconoscere uno tra 30 mila nomi di persona, per cui ogni nome compare una volta sola nelle 120 mila chiamate.

La perplexity della sequenza di 120 mila chiamate si ottiene moltiplicando 120 mila probabilità, di cui 90 mila valgono $1/4$ e 30 mila valgono $1/120\,000$, e prendendo la radice 120 millesima dell'inverso. Il calcolo si semplifica in:

$$\mathrm{PP} = \left(\frac{1}{4} \cdot \frac{1}{4} \cdot \frac{1}{4} \cdot \frac{1}{120\,000}\right)^{-\frac{1}{4}} = 52.6$$

che è molto meno dei 30 mila nomi possibili, perché tre quarti delle chiamate sono facili da predire.

Una perplexity più bassa indica un modello migliore. Addestrando modelli unigram, bigram e trigram su 38 milioni di parole del Wall Street Journal e calcolando la perplexity su un test set di 1.5 milioni di parole si ottiene:

Ordine dell'N-gram	Unigram	Bigram	Trigram
Perplexity	962	170	109

Più contesto usa il modello, meno è sorpreso dalla parola successiva. Due perplexity sono però confrontabili solo se i modelli usano lo stesso vocabolario, e un miglioramento della perplexity non garantisce un miglioramento nel task finale: quando possibile, va confermato con una valutazione estrinseca.

3.5 Generalizzazione e zeri

3.5.1 Il metodo di visualizzazione di Shannon

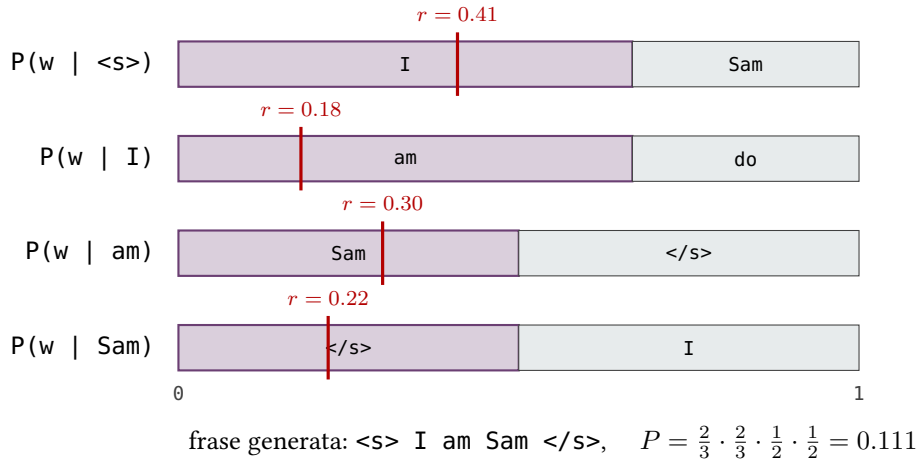
Un modo per capire che tipo di conoscenza contiene un modello di linguaggio è **campionare** frasi da esso. Con i bigrammi la procedura è questa:

1. scegliamo un bigramma casuale (`<s>`, w) secondo la sua probabilità;

2. scegliamo un bigramma casuale (w, x) secondo la sua probabilità, e così via finché non scegliamo `</s>`;

3. mettiamo insieme le parole.

Ad esempio, da `<s> I`, `I want`, `want to`, `to eat`, `eat Chinese`, `Chinese food`, `food </s>` si ottiene "I want to eat Chinese food". Per scegliere una parola "secondo la sua probabilità" immaginiamo che le parole che possono seguire il contesto coprano l'intervallo $[0, 1]$, ognuna con un sotto-intervallo lungo quanto la sua probabilità: estraiamo un numero a caso tra 0 e 1 e scegliamo la parola il cui intervallo lo contiene (figura 3.1).



frase generata: `<s> I am Sam </s>`, $P = \frac{2}{3} \cdot \frac{2}{3} \cdot \frac{1}{2} \cdot \frac{1}{2} = 0.111$

Figura 3.1. Il metodo di Shannon con il modello a bigrammi del mini corpus `I am Sam`, `Sam I am`, `I do not like green eggs and ham`. A ogni passo le parole che possono seguire il contesto si dividono l'intervallo $[0, 1]$ in proporzione alla loro probabilità, e il numero estratto r sceglie la parola (evidenziata). La frase si ferma quando esce `</s>`.

Il campionamento mostra bene quanto il modello dipenda dal corpus di training. Con modelli addestrati sulle opere di Shakespeare, le frasi generate dagli unigrammi non hanno alcuna coerenza, quelle dei bigrammi hanno una coerenza locale tra parole vicine, e quelle dei trigrammi e dei 4-grammi iniziano ad assomigliare molto a Shakespeare.

Anche troppo: la frase "It cannot be but so", generata dal modello a 4-grammi, è presa direttamente dal *King John*. Il motivo è che Shakespeare, come corpus, è piccolo: ha $N = 884\,647$ token e $V = 29\,066$ type, e ha prodotto circa 300 mila tipi di bigrammi su $V^2 \approx 844$ milioni di bigrammi possibili. Il 99.96% dei bigrammi possibili non è mai stato visto, e con i 4-grammi va ancora peggio: quello che esce sembra Shakespeare perché *è* Shakespeare.

3.5.2 Il pericolo dell'overfitting

Gli N-gram predicono bene le parole solo se il corpus di test assomiglia al corpus di training, e nella realtà spesso non è così. Un modello addestrato su Shakespeare e uno addestrato sul Wall Street Journal generano frasi che "sembrano inglese" entrambe, ma che non hanno quasi niente in comune. Dobbiamo quindi addestrare modelli che generalizzino, e un tipo di generalizzazione riguarda gli **zeri**: sequenze che non compaiono mai nel training set ma compaiono nel test set. Supponiamo ad esempio che il training set contenga

... denied the allegations *... denied the reports* *... denied the claims* *... denied the request*

e che nel test set compaiano invece *... denied the offer* e *... denied the loan.* La stima MLE dà $P(\text{offer} \mid \text{denied the}) = 0$. I bigrammi con probabilità zero sono un problema doppio: sottostimano la probabilità di sequenze perfettamente plausibili, e fanno sì che l'intero test set abbia probabilità 0, per cui non possiamo nemmeno calcolare la perplexity (non si può dividere per 0).

Parole sconosciute. Un caso particolare di zero sono le parole mai viste. Se conosciamo in anticipo tutte le parole, il vocabolario V è fisso e si parla di task a **vocabolario chiuso**. Spesso però non è così, e nel test compaiono parole **fuori vocabolario** (out of vocabulary, OOV): si parla allora di task a **vocabolario aperto**.

La soluzione classica è un token speciale `<UNK>`: si fissa un lessico L di dimensione V, in fase di normalizzazione si sostituisce con `<UNK>` ogni parola del training che non è in L, e si stimano le probabilità di `<UNK>` come quelle di una parola normale. In fase di decodifica, ogni parola mai vista prende le probabilità di `<UNK>`[3].

3.6 Smoothing

L'intuizione dello **smoothing** (o *discounting*) è che, quando le statistiche sono sparse, conviene "rubare" un po' di massa di probabilità agli eventi visti per darla a quelli mai visti, e generalizzare meglio. Torniamo all'esempio di *denied the*: nel training abbiamo visto 3 volte *allegations*, 2 volte *reports*, una volta *claims* e una volta *request*, per un totale di 7. Invece di $3/7$, $2/7$, $1/7$ e $1/7$, possiamo usare ad esempio $2.5/7$, $1.5/7$, $0.5/7$ e $0.5/7$, e lasciare $2/7$ a tutte le altre parole (*attack*, *man*, *outcome*, ...).

3.6.1 Laplace smoothing

Il modo più semplice di fare smoothing è il **Laplace smoothing** (o add-one): facciamo finta di aver visto ogni parola una volta in più di quante l'abbiamo vista davvero, ovvero aggiungiamo 1 a tutti i conteggi. Dato che ognuna delle $|V|$ parole del vocabolario riceve un conteggio in più, anche il denominatore deve crescere di $|V|$:

$$P_{\text{MLE}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}w_i)}{C(w_{i-1})} \qquad \longrightarrow \qquad P_{\text{Add-1}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}w_i)+1}{C(w_{i-1})+|V|}. \tag{3.7}$$

Una generalizzazione è l'**add-k smoothing**, in cui si aggiunge un conteggio frazionario k (0.5? 0.01?) al posto di 1, e il denominatore diventa $C(w_{i-1}) + k|V|$. Il valore di k va scelto, ad esempio, sul devset. La figura 3.2 mostra l'effetto sull'esempio di *denied the*.

Applicando il Laplace smoothing ai bigrammi del BeRP (con $|V| = 1446$) cambia tutto. Per vedere di quanto, conviene ricostruire i **conteggi ricostituiti** C^*, ovvero i conteggi che, divisi per il denominatore originale $C(w_{i-1})$, darebbero la probabilità smussata:

$$C^*_{\text{Add-1}}(w_{i-1}w_i) = \frac{C(w_{i-1}w_i)+1}{C(w_{i-1})+|V|} \cdot C(w_{i-1}). \tag{3.8}$$

[3] Con la tokenizzazione in sottoparole (sezione 2.3) il problema scompare: qualsiasi parola si può scrivere come sequenza di token noti, al limite di singole lettere, e il test set non contiene mai token sconosciuti.

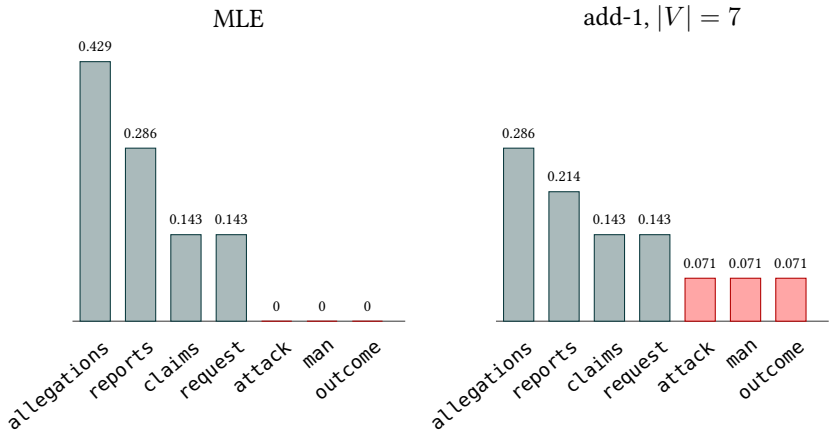


Figura 3.2. Add-one smoothing su $P(w \mid \text{denied the})$, con un vocabolario di sole 7 parole. A sinistra la stima MLE, in cui *attack*, *man* e *outcome* hanno probabilità zero. A destra le probabilità smussate: ogni parola mai vista riceve $1/14$, e alle parole viste resta meno massa. Con il vocabolario del BeRP, $|V| = 1446$, le parole mai viste si prenderebbero il 99% della massa.

Confrontando la tabella 3.3 con la tabella 3.1 si vede che $C(\text{want to})$ passa da 608 a 238, e allo stesso modo $P(\text{to} \mid \text{want})$ passa da 0.66 a 0.26. Il problema è che **troppa massa di probabilità viene spostata sugli zeri**.

	i	want	to	eat	chinese	food	lunch	spend
i	3.8	527	0.64	6.4	0.64	0.64	0.64	1.9
want	1.2	0.39	238	0.78	2.7	2.7	2.3	0.78
to	1.9	0.63	3.1	430	1.9	0.63	4.4	133
eat	0.34	0.34	1	0.34	5.8	1	15	0.34
chinese	0.2	0.098	0.098	0.098	0.098	8.2	0.2	0.098
food	6.9	0.43	6.9	0.43	0.86	2.2	0.43	0.43
lunch	0.57	0.19	0.19	0.19	0.19	0.38	0.19	0.19
spend	0.32	0.16	0.32	0.16	0.16	0.16	0.16	0.16

Tabella 3.3. Conteggi ricostituiti dopo l'add-one smoothing dei bigrammi del BeRP: rispetto alla tabella 3.1 i conteggi alti si riducono di molto (*want to* passa da 608 a 238), mentre gli zeri diventano numeri piccoli ma positivi.

L'add-one smoothing è quindi uno strumento grezzo, e non si usa per gli N-gram: vedremo metodi migliori. Si usa però per fare smoothing in altri modelli di NLP, come la classificazione del testo (capitolo 4), e in generale nei domini in cui gli zeri non sono così tanti.

3.6.2 Backoff e interpolazione

A volte conviene usare **meno contesto**, ovvero condizionare su un contesto più breve per i contesti di cui sappiamo poco. Le strategie sono due:

- **Backoff**: usiamo il trigramma se ne abbiamo abbastanza evidenza, altrimenti il bigramma, altrimenti l'unigramma.
- **Interpolazione**: mescoliamo sempre unigrammi, bigrammi e trigrammi.

In generale l'interpolazione funziona meglio. Nell'**interpolazione lineare** semplice la probabilità del trigramma è una media pesata delle tre stime:

$$\hat{P}(w_i \mid w_{i-2}w_{i-1}) = \lambda_1 P(w_i \mid w_{i-2}w_{i-1}) + \lambda_2 P(w_i \mid w_{i-1}) + \lambda_3 P(w_i), \qquad \sum_j \lambda_j = 1 \quad (3.9)$$

dove il vincolo sulla somma dei λ_j garantisce che il risultato sia ancora una probabilità. In una versione più sofisticata, i pesi dipendono dal contesto, $\lambda_j = \lambda_j(w_{i-2}w_{i-1})$: se abbiamo conteggi affidabili per un certo bigramma, possiamo fidarci di più dei trigrammi che lo estendono e dar loro un peso più alto.

Come scegliere i λ? Si usa un **corpus held-out**, ovvero una parte dei dati tenuta da parte rispetto al training (e distinta dal test set). Si fissano le probabilità degli N-gram stimate sul training, e si cercano i λ che danno la probabilità più alta al corpus held-out. Si può usare l'algoritmo Expectation-Maximization: si fissa λ_1 e si cerca λ_2, con $\lambda_3 = 1 - \lambda_1 - \lambda_2$, poi fissa λ_2 e cerca λ_1, e così via fino a convergenza.

3.6.3 N-gram su scala web

Con corpora enormi come Google N-grams servono accorgimenti specifici. Per lo spazio si fa **pruning**: si memorizzano solo gli N-gram con conteggio sopra una soglia, si eliminano i singleton degli N-gram di ordine più alto, oppure si usa un pruning basato sull'entropia. Per l'efficienza si usano strutture dati come i *trie* e i *Bloom filter* (che danno modelli di linguaggio approssimati). Inoltre si memorizzano le parole come indici e non come stringhe, si usa la codifica di Huffman per far stare molte parole in due byte, e si quantizzano le probabilità con 4–8 bit invece di 8 byte.

Per lo smoothing, su questa scala si usa spesso lo **stupid backoff** [5], che rinuncia a fare discounting e usa direttamente le frequenze relative:

$$S(w_i \mid w_{i-k+1:i-1}) = \begin{cases} \dfrac{C(w_{i-k+1:i})}{C(w_{i-k+1:i-1})} & \text{se } C(w_{i-k+1:i}) > 0 \\ 0.4 \cdot S(w_i \mid w_{i-k+2:i-1}) & \text{altrimenti} \end{cases} \quad (3.10)$$

dove la ricorsione termina sull'unigramma, con $S(w_i) = C(w_i)/N$. Il risultato non è una vera distribuzione di probabilità (per questo si scrive S e non P), ma con dati così abbondanti funziona bene.

3.6.4 Absolute discounting

Supponiamo di voler sottrarre un po' di massa a un conteggio di 4, per darla agli zeri. Quanto dobbiamo sottrarre? L'idea di Church e Gale [7] è stata dividere 22 milioni di parole di AP Newswire in un training set e un held-out set, e per ogni bigramma del training set guardare il suo conteggio reale nell'held-out set. Il risultato è in tabella 3.4.

Tranne che per i conteggi 0 e 1, il conteggio nell'held-out set è circa quello del training meno 0.75. L'**absolute discounting** formalizza questa osservazione: sottrae una quantità fissa d (ad esempio 0.75, magari con un valore diverso per i conteggi 1 e 2) da ogni conteggio, e interpola

Conteggio nel training	Conteggio nell'held-out
0	0.0000270
1	0.448
2	1.25
3	2.24
4	3.23
5	4.21
6	5.23
7	6.21
8	7.21
9	8.26

Tabella 3.4. Per i bigrammi che compaiono da 0 a 9 volte nel training set, il conteggio medio nell'held-out set (dati di Church e Gale su AP Newswire): tranne che per 0 e 1, vale circa $C^* = C - 0.75$.

con l'unigramma:

$$P_{\text{AD}}(w_i \mid w_{i-1}) = \frac{C(w_{i-1}w_i) - d}{C(w_{i-1})} + \lambda(w_{i-1})\, P(w_i) \tag{3.11}$$

dove il primo termine è il bigramma scontato e $\lambda(w_{i-1})$ è il peso dell'interpolazione. Il discount non cambia molto i conteggi alti, che sono già stime affidabili, e modifica soprattutto quelli bassi, di cui comunque non ci fidiamo.

3.6.5 Kneser-Ney smoothing

Resta un dubbio: è giusto usare l'unigramma $P(w_i)$ così com'è? Consideriamo lo Shannon game "I can't see without my reading …". La parola giusta è *glasses*, ma in un corpus di notizie *Kong* può essere più frequente, perché *Hong Kong* compare spesso. Il punto è che *Kong* segue quasi sempre *Hong*, mentre l'unigramma ci serve proprio quando **non** abbiamo visto il bigramma.

Invece di chiederci quanto è probabile w, il **Kneser-Ney smoothing** [13] si chiede quanto è probabile che w compaia come **continuazione nuova**. Per ogni parola contiamo quanti tipi di bigrammi diversi completa, visto che ogni tipo di bigramma è stato una continuazione nuova la prima volta che è stato visto:

$$P_{\text{CONTINUATION}}(w_i) \propto \left|\{w_{i-1} : C(w_{i-1}w_i) > 0\}\right|. \tag{3.12}$$

Per ottenere una probabilità normalizziamo per il numero totale di tipi di bigrammi:

$$P_{\text{CONTINUATION}}(w_i) = \frac{\left|\{w_{i-1} : C(w_{i-1}w_i) > 0\}\right|}{\left|\{(w'_{i-1}, w'_i) : C(w'_{i-1}w'_i) > 0\}\right|} \tag{3.13}$$

o, in modo equivalente, per il numero di parole che precedono tutte le parole, $\sum_{w'_i} |\{w'_{i-1} : C(w'_{i-1}w'_i) > 0\}|$. Una parola frequente come *Kong*, che compare in un solo contesto, ha quindi una probabilità di continuazione bassa.

La formula dell'**interpolated Kneser-Ney** per i bigrammi è:

$$P_{\text{KN}}(w_i \mid w_{i-1}) = \frac{\max(C(w_{i-1}w_i) - d, 0)}{C(w_{i-1})} + \lambda(w_{i-1})\, P_{\text{CONTINUATION}}(w_i) \tag{3.14}$$

dove λ è una costante di normalizzazione, che distribuisce la massa di probabilità scontata:

$$\lambda(w_{i-1}) = \frac{d}{C(w_{i-1})} \cdot |\{w : C(w_{i-1}w) > 0\}|\,. \tag{3.15}$$

Il primo fattore è il discount normalizzato, il secondo è il numero di tipi di parole che possono seguire w_{i-1}, ovvero il numero di parole a cui abbiamo applicato il discount.

La formulazione generale è ricorsiva:

$$P_{\text{KN}}(w_i \mid w_{i-k+1:i-1}) = \frac{\max(c_{\text{KN}}(w_{i-k+1:i}) - d, 0)}{c_{\text{KN}}(w_{i-k+1:i-1})} + \lambda(w_{i-k+1:i-1})\, P_{\text{KN}}(w_i \mid w_{i-k+2:i-1}) \tag{3.16}$$

dove $c_{\text{KN}}(\cdot)$ è il conteggio normale per l'ordine più alto, e il **continuation count** per gli ordini più bassi, ovvero il numero di contesti distinti di una parola in cui compare la sequenza[4].

Riepilogo sullo smoothing. L'add-1 smoothing va bene per la classificazione del testo, ma non per i modelli di linguaggio. Per N-gram molto grandi, come quelli del web, si usa lo stupid backoff. Il metodo più usato in generale è l'interpolated Kneser-Ney, nella sua versione estesa.

3.6.6 Modelli di linguaggio avanzati

Accenniamo infine ad alcune varianti dei modelli N-gram. I **modelli discriminativi** scelgono i pesi degli N-gram per migliorare un task, invece che per adattarsi al training set. Esistono poi modelli basati sul parsing, e i **caching model**, che partono dall'osservazione che le parole usate di recente hanno più probabilità di ricomparire:

$$P_{\text{CACHE}}(w_i \mid \text{history}) = \lambda\, P(w_i \mid w_{i-2}w_{i-1}) + (1-\lambda)\, \frac{C(w_i \in \text{history})}{|\text{history}|}. \tag{3.17}$$

I caching model si sono rivelati però molto poco efficaci per il riconoscimento del parlato.

[4] La ricorsione termina interpolando gli unigrammi con la distribuzione uniforme $1/V$. La versione che funziona meglio è il *modified Kneser-Ney* [6], che usa tre discount diversi per gli N-gram con conteggio 1, 2 e 3 o più.

Capitolo 4

Naive Bayes e classificazione del testo

La **classificazione del testo** consiste nell'assegnare a un testo una classe scelta da un insieme prestabilito. Alcuni esempi:

- **Questa email è spam?**
- **Chi ha scritto i Federalist Papers?** Tra il 1787 e il 1788 Jay, Madison e Hamilton scrissero in forma anonima una serie di saggi per convincere lo stato di New York a ratificare la Costituzione americana. L'attribuzione di 12 di questi saggi è rimasta incerta fino al 1963, quando Mosteller e Wallace [17] la risolsero con metodi bayesiani.
- **Di cosa parla questo articolo medico?** Si tratta di assegnargli una categoria della gerarchia MeSH (Medical Subject Headings), come *Blood Supply*, *Chemistry*, *Drug Therapy* o *Epidemiology*.
- **Questa recensione è positiva o negativa?** "…zany characters and richly applied satire, and some great plot twists" è positiva, "It was pathetic. The worst part about it was the boxing scenes…" è negativa.

L'ultimo è un esempio di **sentiment analysis**, che useremo come esempio guida per tutto il capitolo. Il riferimento per questo capitolo è l'appendice B di [11]. Le applicazioni sono molte: capire se la recensione di un film è positiva o negativa, cosa pensano le persone di un prodotto o come varia la fiducia dei consumatori. Si può anche capire cosa pensano gli elettori di un candidato, fino a predire l'esito di un'elezione o l'andamento del mercato.

Stati affettivi. Secondo la tipologia di Scherer [20], gli stati affettivi sono di diversi tipi:

- **Emozioni**: reazioni brevi e sincronizzate alla valutazione di un evento importante (arrabbiato, triste, gioioso, impaurito, orgoglioso).
- **Umori**: cambiamenti diffusi, senza una causa precisa, di bassa intensità e lunga durata (allegro, cupo, irritabile, depresso).
- **Atteggiamenti interpersonali**: l'atteggiamento verso un'altra persona in una specifica interazione (amichevole, distante, freddo, caloroso, sprezzante).
- **Attitudini**: convinzioni durature e affettivamente connotate verso oggetti o persone (gradire, amare, odiare, apprezzare, desiderare).

- **Tratti della personalità**: disposizioni stabili e comportamenti tipici (nervoso, ansioso, sconsiderato, ostile, geloso).

La sentiment analysis è il rilevamento delle **attitudini**. Nel caso più semplice, che è quello di questo capitolo, ci chiediamo solo se l'attitudine espressa in un testo è positiva o negativa. Oltre alla sentiment analysis, la classificazione del testo comprende la spam detection, l'attribuzione dell'autore, l'identificazione della lingua e l'assegnazione di categorie, argomenti o generi.

4.1 Classificazione del testo

Formalmente, il problema è questo:

- **Input**: un documento d e un insieme fissato di classi $C = \{c_1, c_2, \ldots, c_J\}$.
- **Output**: una classe predetta $c \in C$.

Il primo metodo possibile sono le **regole scritte a mano**, basate su combinazioni di parole o di altre feature. Ad esempio, per lo spam: *indirizzo in blacklist* OR ("dollars" AND "you have been selected"). L'accuratezza può essere alta, se le regole vengono raffinate con cura da un esperto, ma costruirle e mantenerle è costoso, e sono fragili quando i dati cambiano nel tempo.

Il metodo più comune è invece il **machine learning supervisionato**. In questo caso l'input comprende, oltre all'insieme di classi C, un training set di m documenti etichettati a mano $(d_1, c_1), \ldots, (d_m, c_m)$, e l'output è un classificatore appreso $\gamma : d \to c$. Si può usare qualsiasi tipo di classificatore: Naive Bayes, regressione logistica (capitolo 5), reti neurali, k-Nearest Neighbors, e così via.

4.2 Il classificatore Naive Bayes

Naive Bayes è un metodo di classificazione semplice ("naive", ingenuo) basato sulla regola di Bayes. Si appoggia a una rappresentazione molto semplice del documento, la **bag of words**: il testo viene trattato come un insieme non ordinato di parole, di cui si ignora la posizione e si conserva solo la frequenza (figura 4.1).

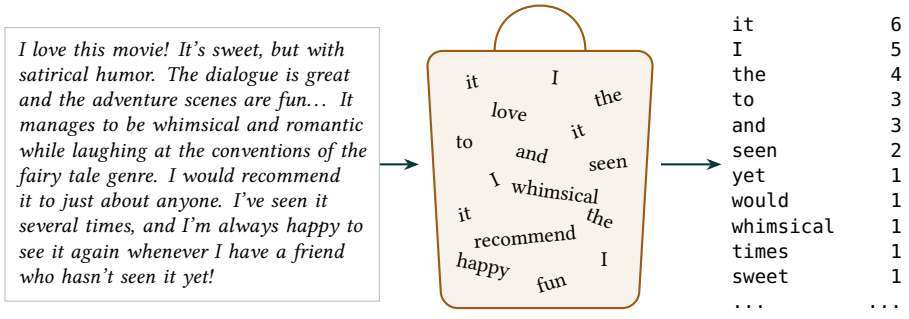


Figura 4.1. La rappresentazione bag of words di una recensione: le parole vengono messe in un "sacchetto" perdendo il loro ordine, e del documento resta solo il conteggio di ciascuna parola.

4.2.1 La regola di Bayes applicata ai documenti

Per un documento d e una classe c, la regola di Bayes dice che:

$$P(c \mid d) = \frac{P(d \mid c)\, P(c)}{P(d)}. \tag{4.1}$$

Il classificatore sceglie la classe più probabile dato il documento, ovvero la classe **MAP** (maximum a posteriori):

$$c_{\text{MAP}} = \operatorname*{argmax}_{c \in C} P(c \mid d) = \operatorname*{argmax}_{c \in C} \frac{P(d \mid c)\, P(c)}{P(d)} = \operatorname*{argmax}_{c \in C} P(d \mid c)\, P(c) \tag{4.2}$$

dove nell'ultimo passaggio abbiamo eliminato il denominatore, che è lo stesso per tutte le classi: stiamo sempre confrontando le classi per lo stesso documento d. Il termine $P(c)$ è il **prior** della classe, mentre $P(d \mid c)$ è la **likelihood** del documento[1].

Rappresentiamo ora il documento d con delle feature $x_1, \ldots, x_n$:

$$c_{\text{MAP}} = \operatorname*{argmax}_{c \in C} \underbrace{P(x_1, x_2, \ldots, x_n \mid c)}_{\text{likelihood}}\, \underbrace{P(c)}_{\text{prior}}. \tag{4.3}$$

Il prior è facile da stimare: basta contare quanto spesso compare ogni classe nel corpus. La likelihood invece ha $O(|X|^n \cdot |C|)$ parametri, uno per ogni possibile combinazione di feature e classe, e potrebbe essere stimata solo con un numero enorme di esempi di training.

4.2.2 Le assunzioni di Naive Bayes

Il Naive Bayes multinomiale risolve il problema con due assunzioni:

- **Bag of words**: la posizione delle parole non conta, e *love* ha lo stesso effetto sulla classificazione che sia la prima, la ventesima o l'ultima parola del documento.
- **Indipendenza condizionale**: le probabilità delle feature $P(x_i \mid c)$ sono indipendenti tra loro, data la classe.

La seconda assunzione permette di scrivere la likelihood come un prodotto:

$$P(x_1, \ldots, x_n \mid c) = P(x_1 \mid c) \cdot P(x_2 \mid c) \cdot P(x_3 \mid c) \cdots P(x_n \mid c) \tag{4.4}$$

e quindi il classificatore diventa:

$$c_{\text{NB}} = \operatorname*{argmax}_{c \in C} P(c) \prod_{x \in X} P(x \mid c). \tag{4.5}$$

Per applicarlo alla classificazione del testo, consideriamo come feature le parole del documento, scorrendo tutte le posizioni del documento di test:

$$c_{\text{NB}} = \operatorname*{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(x_i \mid c_j) \tag{4.6}$$

[1] Naive Bayes è un modello **generativo**: possiamo leggere la formula come una descrizione di come viene generato un documento. Prima si sceglie una classe secondo $P(c)$, poi si generano le parole secondo $P(d \mid c)$. Riprenderemo questa distinzione nel capitolo 5.

dove *positions* indica tutte le posizioni delle parole nel documento di test.

Lo spazio dei logaritmi. Moltiplicare tante probabilità, come $0.0006 \cdot 0.0007 \cdot 0.0009 \cdot 0.01 \cdot 0.5 \cdot 0.000008 \cdots$, porta all'underflow. Come per i modelli di linguaggio (capitolo 3), si lavora quindi nello spazio dei logaritmi, sommando i logaritmi delle probabilità invece di moltiplicare le probabilità:

$$c_{\text{NB}} = \underset{c_j \in C}{\operatorname{argmax}} \left[\log P(c_j) + \sum_{i \in \text{positions}} \log P(x_i \mid c_j) \right]. \tag{4.7}$$

Osserviamo due cose. La prima, è che il logaritmo non cambia l'ordine delle classi: la classe con la probabilità più alta ha anche il logaritmo della probabilità più alto. La seconda è che la formula è il massimo di una somma di pesi, ovvero una funzione lineare degli input: Naive Bayes è un **classificatore lineare**.

4.3 Addestrare Naive Bayes

Come primo tentativo usiamo le stime di massima verosimiglianza, ovvero le frequenze nei dati:

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{\text{total}}}, \qquad \hat{P}(w_i \mid c_j) = \frac{\text{count}(w_i, c_j)}{\sum_{w \in V} \text{count}(w, c_j)} \tag{4.8}$$

dove N_{c_j} è il numero di documenti della classe c_j e N_{total} il numero totale di documenti. Per stimare le likelihood si crea, per ogni classe c_j, un "mega-documento" che concatena tutti i documenti di quella classe, e si usa la frequenza di w_i in questo mega-documento: $\hat{P}(w_i \mid c_j)$ è la frazione di volte in cui w_i compare tra tutte le parole dei documenti della classe c_j. Il vocabolario V è l'unione dei type di tutte le classi, e non contiene soltanto quelli della classe c_j.

Il problema della massima verosimiglianza. Cosa succede se nel training non abbiamo mai visto la parola *fantastic* in un documento positivo? La stima è:

$$\hat{P}(\text{fantastic} \mid \text{positive}) = \frac{\text{count}(\text{fantastic}, \text{positive})}{\sum_{w \in V} \text{count}(w, \text{positive})} = 0$$

e dato che nella (4.6) le likelihood si moltiplicano, uno zero annulla la probabilità della classe, qualunque sia il resto dell'evidenza. È lo stesso problema degli zeri dei modelli N-gram, e la soluzione più semplice è la stessa: il **Laplace smoothing** (add-1):

$$\hat{P}(w_i \mid c) = \frac{\text{count}(w_i, c) + 1}{\sum_{w \in V} \big(\text{count}(w, c) + 1\big)} = \frac{\text{count}(w_i, c) + 1}{\big(\sum_{w \in V} \text{count}(w, c)\big) + |V|}. \tag{4.9}$$

Come avevamo anticipato, l'add-1 smoothing non funziona bene per i modelli di linguaggio, ma è molto usato nella classificazione del testo. L'algoritmo 2 riassume l'addestramento e il test, con lo smoothing generalizzato a un valore α qualsiasi.

Parole sconosciute e stop word. Cosa facciamo con le parole che compaiono nel test ma non nel training, e quindi non sono nel vocabolario? Le ignoriamo: le togliamo dal documento di test, come se non ci fossero, senza includere nessuna probabilità per loro. Non serve costruire un

Algoritmo 2 Addestramento e test di Naive Bayes multinomiale con add-α smoothing, nello spazio dei logaritmi.

function TRAINNAIVEBAYES(documenti D, classi C)
    $V \leftarrow$ vocabolario di D
    for all $c_j \in C$ **do**
        $\text{docs}_j \leftarrow$ tutti i documenti di D con classe c_j
        $\text{logprior}[c_j] \leftarrow \log \frac{|\text{docs}_j|}{|D|}$
        $\text{Text}_j \leftarrow$ un unico documento che concatena docs_j
        $n \leftarrow$ numero di token in Text_j
        for all $w_k \in V$ **do**
            $n_k \leftarrow$ numero di occorrenze di w_k in Text_j
            $\text{loglikelihood}[w_k, c_j] \leftarrow \log \frac{n_k+\alpha}{n+\alpha|V|}$
        end for
    end for
    return logprior, loglikelihood, V
end function
function TESTNAIVEBAYES(documento d, logprior, loglikelihood, C, V)
    for all $c_j \in C$ **do**
        $\text{sum}[c_j] \leftarrow \text{logprior}[c_j]$
        for all posizione i in d **do**
            if $d[i] \in V$ **then** ▷ le parole sconosciute si ignorano
                $\text{sum}[c_j] \leftarrow \text{sum}[c_j] + \text{loglikelihood}[d[i], c_j]$
            end if
        end for
    end for
    return $\text{argmax}_{c_j}\ \text{sum}[c_j]$
end function

modello delle parole sconosciute, perché sapere quale classe ha più parole sconosciute in generale non aiuta.

Alcuni sistemi ignorano anche le **stop word**, ovvero le parole molto frequenti come *the* e *a*: si ordina il vocabolario per frequenza nel training set, si prendono le prime 10 o 50 parole come lista di stop word e le si rimuove sia dal training sia dal test. Rimuovere le stop word però di solito non aiuta, e in pratica la maggior parte delle implementazioni di Naive Bayes usa tutte le parole.

4.4 Un esempio svolto

Vediamo un esempio di sentiment analysis con add-1 smoothing, con due classi, positiva ($+$) e negativa ($-$), e questi documenti di training e di test:

	Classe	Documento
Training	$-$	just plain boring
	$-$	entirely predictable and lacks energy
	$-$	no surprises and very few laughs
	$+$	very powerful
	$+$	the most fun film of the summer
Test	?	predictable with no fun

I passi sono quattro, e la figura 4.2 li riassume:

1. **Prior**: tre documenti su cinque sono negativi, quindi $P(-) = 3/5$ e $P(+) = 2/5$.
2. **Parole sconosciute**: *with* non compare nel training set, quindi la togliamo.
3. **Likelihood**: il vocabolario ha $|V| = 20$ parole, i documenti negativi hanno in tutto 14 token e quelli positivi 9. Con la (4.9):

$$
\begin{aligned}
P(\text{predictable} \mid -) &= \frac{1+1}{14+20} & P(\text{predictable} \mid +) &= \frac{0+1}{9+20} \\
P(\text{no} \mid -) &= \frac{1+1}{14+20} & P(\text{no} \mid +) &= \frac{0+1}{9+20} \\
P(\text{fun} \mid -) &= \frac{0+1}{14+20} & P(\text{fun} \mid +) &= \frac{1+1}{9+20}
\end{aligned}
$$

4. **Punteggio**: per la frase di test S = “predictable no fun”:

$$
\begin{aligned}
P(-)\,P(S \mid -) &= \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}, \\
P(+)\,P(S \mid +) &= \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.3 \times 10^{-5}.
\end{aligned}
$$

Il modello predice quindi la classe negativa[2].

> **Esercizio 4.1 (Naive Bayes su IMDB).** Sviluppare un classificatore Naive Bayes e addestrarlo sul dataset di recensioni IMDB (`https://huggingface.co/datasets/stanfordnl`

[2] Il valore esatto è $\frac{2}{5} \cdot \frac{2}{24389} = 3.28 \times 10^{-5}$, che spesso si trova troncato a 3.2×10^{-5}. La conclusione non cambia.

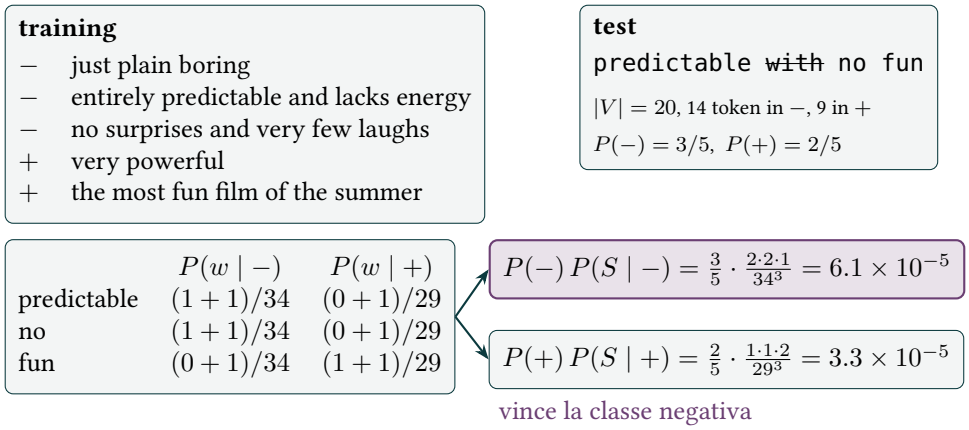


Figura 4.2. L'esempio svolto di Naive Bayes con add-1 smoothing: la parola sconosciuta *with* viene eliminata, le tre parole restanti hanno una likelihood per ciascuna classe, e il punteggio più alto è quello della classe negativa.

`p/imdb`).

4.5 Ottimizzare Naive Bayes per la sentiment analysis

4.5.1 Binary Naive Bayes

Per task come la sentiment analysis, il fatto che una parola compaia sembra contare più della sua frequenza: che in una recensione compaia *fantastic* ci dice molto, che compaia cinque volte non ci dice molto di più. Il **binary multinomial Naive Bayes** (o binary NB) taglia quindi i conteggi delle parole a 1[3]. L'algoritmo è lo stesso dell'algoritmo 2, con una sola modifica: prima di concatenare i documenti di una classe, si eliminano i duplicati in ogni documento, conservando un'unica istanza di ogni type. Allo stesso modo, sul documento di test si eliminano i duplicati prima di applicare la (4.6).

La binarizzazione avviene **dentro il singolo documento**, per cui i conteggi finali possono essere anche maggiori di 1. Consideriamo quattro documenti (senza smoothing, per vedere meglio le differenze):

− it was pathetic the worst part was the boxing scenes
− no plot twists or great scenes
+ and satire and great plot twists
+ great scenes great film

Dopo la binarizzazione, *the* e *was* contano una volta sola nel primo documento, *and* una volta sola nel terzo e *great* una volta sola nel quarto. I conteggi diventano quindi:

[3]Non è la stessa cosa del Bernoulli Naive Bayes, che modella anche l'assenza delle parole.

	and	great	scenes	the	was	plot	twists	film
NB, +	2	3	1	0	0	1	1	1
NB, −	0	1	2	2	2	1	1	0
Binary, +	1	2	1	0	0	1	1	1
Binary, −	0	1	2	1	1	1	1	0

dove *great* ha ancora conteggio 2 nella classe positiva, perché compare in due documenti diversi.

4.5.2 Negazione

Consideriamo "I really like this movie" e "I really don't like this movie": la negazione cambia il significato di *like* da positivo a negativo. Allo stesso modo, la negazione può rendere positivo (o quasi) qualcosa di negativo: "Don't dismiss this film", "Doesn't let us get bored". Una baseline molto semplice [18] consiste nell'aggiungere il prefisso `NOT_` a ogni parola compresa tra una negazione e la punteggiatura successiva:

didn't like this movie , but I $\longrightarrow$ *didn't NOT_like NOT_this NOT_movie , but I*

Le nuove "parole" come `NOT_like` compariranno più spesso nei documenti negativi, mentre parole come `NOT_bored` acquisiranno un'associazione positiva.

4.5.3 Lessici

A volte non abbiamo abbastanza dati etichettati. In questo caso possiamo usare liste di parole già annotate, chiamate **lessici**, e ne esistono diverse disponibili pubblicamente:

- **MPQA Subjectivity Cues Lexicon** [27]: 6885 parole (da 8221 lemmi) annotate per intensità (forte o debole), di cui 2718 positive, come *admirable*, *beautiful*, *confident*, *great*, e 4912 negative, come *awful*, *bad*, *cheat*, *hate*.
- **General Inquirer** [24]: 1915 parole positive e 2291 negative, più altre categorie come forte o debole, attivo o passivo, piacere, dolore, virtù, vizio.

Il modo più comune di usare un lessico è aggiungere una feature che si incrementa ogni volta che compare una parola del lessico, ad esempio "questa parola compare nel lessico positivo" e "questa parola compare nel lessico negativo". Tutte le parole positive (*good*, *great*, *beautiful*, *wonderful*) contano così per la stessa feature. Usare una o due feature non funziona bene quanto usare tutte le parole, ma quando i dati di training sono pochi o non rappresentativi del test set, feature dense come quelle dei lessici possono generalizzare meglio.

4.6 Naive Bayes per altri task

Le feature di Naive Bayes non devono essere per forza parole: possono esprimere qualsiasi proprietà del testo. Nello **spam filtering**, ad esempio, il tool SpamAssassin usa feature come "menziona milioni di dollari" (un'espressione regolare per somme di denaro sospette), "il campo From inizia con molti numeri" o "l'oggetto è tutto in maiuscolo". Altre feature sono "l'HTML ha un rapporto basso tra testo e area delle immagini", la frase "One hundred percent guaranteed" o "dice che puoi essere rimosso dalla lista".

Nell'**identificazione della lingua** (language ID), invece, le feature più efficaci sono gli N-gram di caratteri, ed è opportuno addestrare il modello su molte varietà di ogni lingua (ad esempio sull'African American English, o sull'inglese parlato in India).

In generale, Naive Bayes non è così ingenuo:

- è molto veloce e richiede poco spazio;
- funziona bene anche con pochissimi dati di training;
- è robusto alle feature irrilevanti, che si compensano a vicenda senza influenzare il risultato;
- funziona bene nei domini con molte feature ugualmente importanti, dove ad esempio gli alberi di decisione soffrono di frammentazione, soprattutto con pochi dati;
- è ottimo se le assunzioni di indipendenza sono vere: in quel caso è il classificatore ottimo di Bayes per il problema.

È quindi una baseline affidabile per la classificazione del testo, anche se vedremo altri classificatori con un'accuratezza migliore.

4.7 Naive Bayes come modello di linguaggio

Naive Bayes può usare qualsiasi tipo di feature: URL, indirizzi email, dizionari. Se però usiamo solo le parole come feature, e le usiamo tutte (non un sottoinsieme), Naive Bayes ha una somiglianza importante con i modelli di linguaggio del capitolo 3: **ogni classe è un modello di linguaggio unigram**. Le likelihood assegnano una probabilità $P(w \mid c)$ a ogni parola, e quindi anche una probabilità a ogni frase:

$$P(s \mid c) = \prod_{i \in \text{positions}} P(w_i \mid c). \tag{4.10}$$

Forniamo il seguente esempio. Supponiamo che le likelihood delle due classi siano:

w	$P(w \mid +)$	$P(w \mid -)$
I	0.1	0.2
love	0.1	0.001
this	0.01	0.01
fun	0.05	0.005
film	0.1	0.1

La frase "I love this fun film" ha probabilità $0.1 \times 0.1 \times 0.01 \times 0.05 \times 0.1 = 5 \times 10^{-7}$ secondo il modello positivo e $0.2 \times 0.001 \times 0.01 \times 0.005 \times 0.1 = 10^{-9}$ secondo quello negativo, quindi $P(s \mid +) > P(s \mid -)$. Si tratta solo della likelihood, senza il prior, per cui questo è un approccio a massima verosimiglianza: moltiplicando per il prior, il classificatore completo potrebbe prendere una decisione diversa.

4.8 Valutazione

Consideriamo per ora un task di classificazione binaria. Supponiamo di essere il CEO della *Delicious Pie Company*, e di voler sapere cosa dice la gente delle nostre torte. Costruiamo quindi

un rilevatore di tweet su Delicious Pie, in cui la classe positiva sono i tweet che parlano della Delicious Pie Company e la classe negativa tutti gli altri. Per valutarlo confrontiamo l'output del sistema con le etichette corrette, assegnate a mano (le **gold label**), in una **matrice di confusione** (figura 4.3).

		gold standard labels	
		gold positive	gold negative
system output	system positive	true positive	false positive
	system negative	false negative	true negative

$$\text{precision} = \frac{tp}{tp + fp}$$

$$\text{recall} = \frac{tp}{tp + fn}$$

$$\text{accuracy} = \frac{tp + tn}{tp + fp + tn + fn}$$

Figura 4.3. La matrice di confusione di un classificatore binario: le righe sono l'output del sistema, le colonne le gold label. La precision si calcola sulla riga dei positivi del sistema, la recall sulla colonna dei positivi veri.

Perché non l'accuratezza? L'accuratezza (accuracy) è la percentuale di osservazioni classificate correttamente, e sembrerebbe la metrica più naturale. Supponiamo però di aver raccolto un milione di tweet, di cui solo 100 parlano della Delicious Pie Company e 999 900 di altro. Un classificatore stupido, che etichetta ogni tweet come "non parla di torte", avrebbe un'accuratezza del 99.99%. Eppure sarebbe inutile, perché non troverebbe nessuno dei tweet che stiamo cercando. L'accuratezza non è una buona metrica quando le classi sono sbilanciate, che è proprio il caso in cui vogliamo trovare qualcosa di raro.

Precision e recall. Si usano quindi altre due metriche:

- la **precision** è la percentuale degli elementi rilevati dal sistema (ovvero etichettati come positivi) che sono davvero positivi secondo le gold label: $tp/(tp + fp)$;
- la **recall** è la percentuale degli elementi davvero presenti nell'input che il sistema ha identificato correttamente: $tp/(tp + fn)$.

Il nostro classificatore stupido ha un'accuratezza del 99.99% ma una recall di 0, perché non trova nessuno dei 100 tweet sulle torte. A differenza dell'accuratezza, precision e recall mettono al centro i veri positivi, ovvero le cose che stiamo cercando.

F-measure. Per combinare precision (P) e recall (R) in un solo numero si usa la **F-measure**:

$$F_\beta = \frac{(\beta^2 + 1)PR}{\beta^2 P + R} \tag{4.11}$$

dove β pesa l'importanza delle due metriche: con $\beta > 1$ conta di più la recall, con $\beta < 1$ la precision. Quasi sempre si usa $\beta = 1$, ovvero la **F1**:

$$F_1 = \frac{2PR}{P + R}. \tag{4.12}$$

La F-measure è una media armonica pesata di precision e recall. La media armonica di due valori è più vicina al minimo dei due rispetto alla media aritmetica, quindi è una scelta più conservativa: per avere una F1 alta servono sia la precision sia la recall alte.

4.8.1 Più di due classi

Oltre alla classificazione binaria, con due classi come 0 e 1, positiva e negativa o spam e non spam, ci sono altri due casi:

- **Classificazione multi-classe**: un testo è associato a una tra N classi, ad esempio positiva, negativa o neutra, oppure urgent, normal o spam.
- **Classificazione multi-label**: un testo può essere associato a zero, una o più delle N classi, che quindi non sono mutuamente esclusive. Nell'emotion detection, ad esempio, "What a surprise, I'm happy to see you!" esprime sia gioia sia sorpresa. In questo caso si tratta ogni classe separatamente come un problema binario. L'insieme delle etichette di un testo si rappresenta quindi con un vettore binario (*one-hot encoding*), in cui ogni posizione corrisponde a una classe e vale 1 se il testo è associato a quella classe, 0 altrimenti.

Con più classi la matrice di confusione diventa più grande. Consideriamo un classificatore di email in tre classi (tabella 4.1): la precision di una classe si calcola sulla sua riga e la recall sulla sua colonna.

		gold labels			
		urgent	normal	spam	precision
system output	urgent	8	10	1	$8/(8+10+1)$
	normal	5	60	50	$60/(5+60+50)$
	spam	3	30	200	$200/(3+30+200)$
	recall	$8/(8+5+3)$	$60/(10+60+30)$	$200/(1+50+200)$	

Tabella 4.1. Matrice di confusione per una classificazione in tre classi (urgent, normal, spam): ad esempio il sistema ha etichettato come urgent un'email che in realtà era spam.

Per ottenere un'unica metrica le possibilità sono due:

- **Macroaveraging**: si calcolano le prestazioni per ogni classe, e poi si fa la media sulle classi.
- **Microaveraging**: si raccolgono le decisioni di tutte le classi in un'unica matrice di confusione, e da questa si calcolano precision e recall.

Nell'esempio, separando le classi si ottengono le matrici della tabella 4.2.

La macroaverage precision è $(.42 + .52 + .86)/3 = .60$, mentre la microaverage precision è $268/(268 + 99) = .73$. Il microaverage è dominato dalla classe più frequente (qui spam), perché i conteggi vengono sommati. Il macroaverage riflette meglio le prestazioni sulle classi piccole, ed è quindi più opportuno quando tutte le classi sono ugualmente importanti. In Python tutte queste metriche si calcolano con la funzione `classification_report` di scikit-learn.

	Urgent		Normal		Spam		Pooled	
	true	not	true	not	true	not	true	not
system yes	8	11	60	55	200	33	268	99
system no	8	340	40	212	51	83	99	635
precision	$\frac{8}{8+11} = .42$		$\frac{60}{60+55} = .52$		$\frac{200}{200+33} = .86$		$\frac{268}{268+99} = .73$	

Tabella 4.2. Le matrici di confusione delle tre classi della tabella 4.1, prese una alla volta, e la loro somma (pooled). La macroaverage precision è $(.42 + .52 + .86)/3 = .60$, la microaverage precision è quella della matrice pooled, .73.

4.9 Development set e cross-validation

Come per i modelli di linguaggio, si addestra sul training set, si regolano i parametri sul **development set** (devset) e si riportano le prestazioni sul test set. In questo modo si evita l'overfitting sul test set (*tuning to the test set*), e si ottiene una stima più conservativa delle prestazioni. C'è però un paradosso: vorremmo usare più dati possibile per il training, ma anche per il devset. Come dividiamo i dati?

La soluzione è la **cross-validation**. Si sceglie un numero k e si partizionano i dati in k sottoinsiemi disgiunti, detti **fold**. Si usa uno dei fold come devset e si addestra il classificatore sugli altri $k-1$, poi si ripete con un altro fold, per k volte, e alla fine si mettono insieme (pool) i risultati dei k run. Con $k = 10$ si parla di **10-fold cross-validation**.

Visto che in questo modo tutti i dati vengono usati per valutare, non potremmo guardarli per scegliere le feature senza "sbirciare" il test set: per questo si tiene comunque da parte un test set fisso, e si fa la cross-validation solo dentro il training set (figura 4.4).

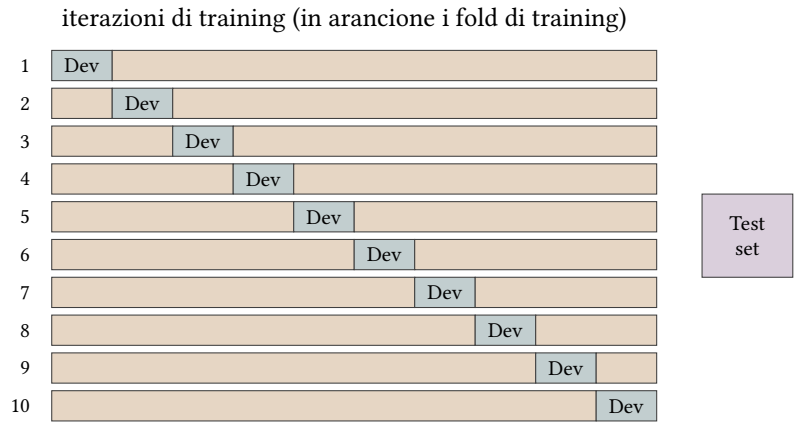


Figura 4.4. 10-fold cross-validation dentro il training set: a ogni iterazione un fold diverso fa da devset e gli altri nove da training. Il test set resta fuori, e si usa una volta sola alla fine.

4.10 Significatività statistica

Come facciamo a sapere se un classificatore è migliore di un altro? Supponiamo di avere due classificatori A e B e una metrica M, dove $M(A, x)$ è la prestazione di A sul test set x. Indichiamo

con $\delta(x)$ la differenza di prestazione tra A e B su x:

$$\delta(x) = M(A, x) - M(B, x) \tag{4.13}$$

dove $\delta(x)$ è detto **effect size**. Vogliamo sapere se $\delta(x) > 0$, ovvero se A è migliore di B. Se guardiamo $\delta(x)$ e vediamo che è positivo, abbiamo finito? No: potrebbe essere solo un caso, dovuto a questo particolare test set o alle circostanze dell'esperimento.

4.10.1 Test di ipotesi

Consideriamo due ipotesi:

- l'**ipotesi nulla** H_0: A non è migliore di B, ovvero $\delta(x) \leq 0$;
- l'ipotesi alternativa H_1: A è migliore di B, ovvero $\delta(x) > 0$.

Vogliamo escludere H_0. Creiamo quindi una variabile aleatoria X che varia su tutti i possibili test set, e ci chiediamo: se H_0 fosse vera, quanto sarebbe probabile vedere, tra questi test set, il $\delta(x)$ che abbiamo visto? Questa probabilità è il **p-value**:

$$P\big(\delta(X) \geq \delta(x) \mid H_0 \text{ è vera}\big). \tag{4.14}$$

Se H_0 è vera ma $\delta(x)$ è enorme, siamo sorpresi, e il p-value è molto basso. Un p-value molto piccolo (le soglie usuali sono .05 o .01) vuol dire che la differenza osservata è molto improbabile sotto l'ipotesi nulla, e possiamo quindi rifiutarla. Un risultato (ad esempio "A è migliore di B") è **statisticamente significativo** se il δ osservato ha una probabilità sotto la soglia, e quindi rifiutiamo l'ipotesi nulla.

Come si calcola questa probabilità? In NLP non si usano di solito test parametrici, come il t-test, perché fanno assunzioni sulla distribuzione della statistica (ad esempio la normalità) che in genere non valgono. Si usano invece **test non parametrici basati sul campionamento**: si creano artificialmente molte versioni dell'esperimento. Supponiamo, ad esempio, di avere moltissimi test set x': misurando $\delta(x')$ su ciascuno otteniamo una distribuzione. Fissata una soglia (ad esempio .01), se vediamo che nel 99% dei test set $\delta(x) > \delta(x')$, concludiamo che il $\delta(x)$ osservato è reale e non un artefatto.

I due approcci più comuni sono l'**approximate randomization** e il **bootstrap test**, entrambi nella versione **paired**, in cui si confrontano due insiemi di osservazioni accoppiate: confrontando A e B sullo stesso test set, possiamo confrontare le loro prestazioni sulla stessa osservazione x_i.

4.10.2 Il paired bootstrap test

Il **bootstrap test** [8] si può applicare a qualsiasi metrica (accuracy, precision, recall, F1, ...). Il termine *bootstrap* indica l'estrazione ripetuta, con reinserimento, di molti campioni (detti **bootstrap sample**) da un campione originale più grande. L'unica assunzione è che il campione originale sia rappresentativo della popolazione.

Forniamo il seguente esempio: un test set x di 10 documenti, con l'accuratezza come metrica. Ogni documento ha uno di quattro esiti: A e B corretti, A e B sbagliati, A corretto e B sbagliato, A sbagliato e B corretto. A ha un'accuratezza di .70 e B di .50, quindi $\delta(x) = .20$. Creiamo ora un gran numero b, ad esempio $10\,000$, di test set virtuali $x^{(i)}$, ognuno di dimensione $n = 10$:

per costruire ciascuno estraiamo a caso una cella della riga x, con reinserimento, per 10 volte (figura 4.5).

Ora abbiamo una distribuzione, e possiamo controllare quanto spesso A ha un vantaggio accidentale. Assumendo H_0 ci aspetteremmo $\delta(x') = 0$, quindi potremmo contare quante volte il $\delta(x^{(i)})$ trovato supera il valore atteso 0 di $\delta(x)$ o più:

$$\text{p-value}(x) = \frac{1}{b}\sum_{i=1}^{b} \mathbf{1}\big(\delta(x^{(i)}) - \delta(x) \geq 0\big) \tag{4.15}$$

dove $\mathbf{1}(\cdot)$ vale 1 se la condizione è soddisfatta e 0 altrimenti. In realtà è un po' più complicato: i campioni non vengono da una distribuzione con media 0, ma dal test set originale x, che è sbilanciato di .20 a favore di A. Per misurare quanto è sorprendente il $\delta(x)$ osservato contiamo quindi quante volte $\delta(x^{(i)})$ supera il valore atteso $\delta(x)$ di $\delta(x)$ o più:

$$\text{p-value}(x) = \frac{1}{b}\sum_{i=1}^{b} \mathbf{1}\big(\delta(x^{(i)}) - \delta(x) \geq \delta(x)\big) = \frac{1}{b}\sum_{i=1}^{b} \mathbf{1}\big(\delta(x^{(i)}) \geq 2\delta(x)\big). \tag{4.16}$$

Supponiamo ad esempio di avere 10 000 test set virtuali e una soglia di .01, e che solo in 47 di essi si abbia $\delta(x^{(i)}) \geq 2\delta(x)$. Il p-value è .0047, più piccolo di .01: il $\delta(x)$ osservato è abbastanza sorprendente, rifiutiamo l'ipotesi nulla e concludiamo che A è migliore di B. Con i 10 documenti del nostro esempio, invece, il risultato è molto diverso: la figura 4.5 mostra che il p-value è circa .27, e la differenza di accuratezza non è significativa[4].

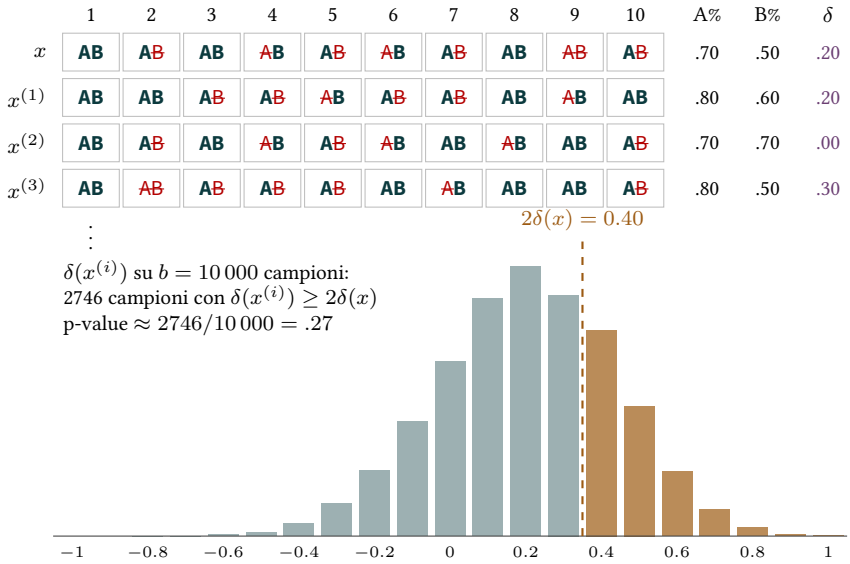


Figura 4.5. Il paired bootstrap test sul test set x di 10 documenti: A e B sono in rosso e barrati quando sbagliano. Ogni test set virtuale $x^{(i)}$ si ottiene estraendo 10 celle di x con reinserimento. Sotto, l'istogramma di $\delta(x^{(i)})$ su 10 000 campioni: quelli con $\delta(x^{(i)}) \geq 2\delta(x) = 0.40$ (in arancione) sono il 27%, quindi il vantaggio di A non è significativo.

[4] Nell'algoritmo 3 la condizione è stretta, $\delta(x^{(i)}) > 2\delta(x)$, mentre nella formula del p-value è $\geq$. Con un test set così piccolo δ assume pochi valori, e la differenza si vede: con la condizione stretta il p-value scende a circa .15, comunque sopra la soglia.

L'algoritmo completo, nella versione di Berg-Kirkpatrick et al. [3], è l'algoritmo 3. Una sua implementazione è disponibile nel pacchetto Python `boostsa` [9].

Algoritmo 3 Il paired bootstrap test.

function BOOTSTRAP(test set x, numero di campioni b)
  calcola $\delta(x)$ ▷ di quanto A è migliore di B su x
  $s \leftarrow 0$
  for $i = 1$ to b **do**
    for $j = 1$ to n **do** ▷ estrae un bootstrap sample $x^{(i)}$ di dimensione n
      scegli un elemento di x a caso e aggiungilo a $x^{(i)}$
    end for
    calcola $\delta(x^{(i)})$ ▷ di quanto A è migliore di B su $x^{(i)}$
    $s \leftarrow s + 1$ se $\delta(x^{(i)}) > 2\delta(x)$
  end for
  p-value$(x) \approx s/b$ ▷ in che percentuale dei campioni A ha battuto le attese?
  return p-value(x) ▷ se sono pochissimi, il δ osservato non è un caso
end function

4.11 Danni nei classificatori

I classificatori possono causare danni concreti. Kiritchenko e Mohammad [12] hanno trovato che la maggior parte dei classificatori di sentiment assegna un sentiment più basso, ed emozioni più negative, alle frasi che contengono nomi afroamericani, perpetuando gli stereotipi che associano gli afroamericani alle emozioni negative.

Allo stesso modo, la **toxicity detection** è il task di rilevare hate speech, abusi, molestie o altri tipi di linguaggio tossico. Alcuni classificatori di tossicità, però, segnalano come tossiche frasi che non lo sono e che si limitano a menzionare delle identità, come le persone cieche, le donne o le persone gay. Questo può portare a censurare le discussioni su questi gruppi.

Questi danni possono avere diverse cause. Possono esserci problemi nei dati di training (è noto che i sistemi di machine learning amplificano i bias dei dati), nelle etichette assegnate dagli annotatori o nelle risorse usate, come i lessici. Oppure il problema può essere nell'architettura del modello, ad esempio in cosa il modello è addestrato a ottimizzare. La mitigazione di questi danni è un'area di ricerca aperta. Nel frattempo, una buona pratica è rilasciare ogni modello con una **model card** [16], che documenta:

- gli algoritmi e i parametri di training;
- le fonti dei dati di training, le motivazioni e il pre-processing;
- le fonti dei dati di valutazione, le motivazioni e il pre-processing;
- l'uso e gli utenti previsti;
- le prestazioni del modello su diversi gruppi demografici e in diverse situazioni.

Capitolo 5

Regressione logistica

La **regressione logistica** è uno strumento di analisi molto usato nelle scienze naturali e sociali, ed è la baseline del machine learning supervisionato per la classificazione. È anche la base delle reti neurali: come vedremo più avanti nel corso, una rete neurale si può vedere come una serie di classificatori logistici messi uno sopra l'altro. Il riferimento per questo capitolo è il capitolo 4 di [11]. Vediamo come si classifica un testo con la regressione logistica, come si addestra il modello con la cross-entropy loss e la discesa del gradiente, e come si estende il modello a più di due classi.

5.1 Classificatori generativi e discriminativi

Naive Bayes (capitolo 4) è un classificatore **generativo**, mentre la regressione logistica è un classificatore **discriminativo**. Per capire la differenza, supponiamo di voler distinguere le immagini di gatti da quelle di cani:

- un **classificatore generativo** costruisce un modello di cosa c'è nell'immagine di un gatto: conosce i baffi, le orecchie, gli occhi, e assegna a qualsiasi immagine una probabilità ("quanto è gattosa questa immagine?"). Costruisce poi un modello analogo per i cani, e data una nuova immagine esegue entrambi i modelli e guarda quale si adatta meglio;
- un **classificatore discriminativo** cerca solo di distinguere i cani dai gatti. Se si accorge che i cani hanno il collare, può ignorare tutto il resto.

Formalmente, per trovare la classe c di un documento d, Naive Bayes passa per la likelihood e il prior:

$$\hat{c} = \underset{c \in C}{\operatorname{argmax}} \overbrace{P(d \mid c)}^{\text{likelihood}} \overbrace{P(c)}^{\text{prior}}.$$

La regressione logistica calcola invece direttamente $P(c \mid d)$. Impara quindi ad assegnare pesi alti alle feature che aiutano a distinguere le classi, anche se non sarebbe in grado di generare un esempio di nessuna delle due.

5.2 Machine learning supervisionato

Ricordiamo le basi del machine learning supervisionato. L'obiettivo è trovare la funzione f che predice meglio $y = f(x)$ dato un input x. Per trovarla abbiamo a disposizione un insieme di

campioni noti $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(n)}, y^{(n)})\}$, e scegliamo, in un insieme di funzioni possibili $\mathcal{F}$, quella che minimizza l'errore medio:

$$f = \operatorname*{argmin}_{f' \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\big(f'(x^{(i)}), y^{(i)}\big) \tag{5.1}$$

dove la funzione di loss $\mathcal{L}$ quantifica la divergenza tra il dato vero e la predizione.

La scelta di $\mathcal{F}$ è delicata. Se $\mathcal{F}$ è troppo ampio, trovare la f migliore può essere impossibile in pratica, e la f migliore sul training set potrebbe non generalizzare bene su altri dati (**overfitting**). Se $\mathcal{F}$ è troppo ristretto, invece, la f migliore potrebbe non approssimare bene nemmeno il training set. In generale vogliamo scegliere $\mathcal{F}$ in modo che il problema di ottimizzazione sia trattabile, e poi restringerlo ulteriormente guardando quanto bene i risultati generalizzano su altri dati (**tuning**).

Un esempio. Vogliamo predire lo stipendio annuo in euro di una persona a partire dai suoi anni di esperienza. Restringiamo lo spazio delle funzioni a quelle lineari, $\mathcal{F} = \{f \mid f(x) = a + b \cdot x,\ a, b \in \mathbb{R}\}$, e usiamo come loss il valore assoluto dell'errore, $\mathcal{L}(y, \hat{y}) = |y - \hat{y}|$:

$$f = \operatorname*{argmin}_{f' \in \mathcal{F}} \frac{1}{n} \sum_{i=1}^{n} \big|f'(x^{(i)}) - y^{(i)}\big|. \tag{5.2}$$

Restringerci alle funzioni lineari può essere limitante, ma rende il training molto semplice: è una regressione lineare (figura 5.1).

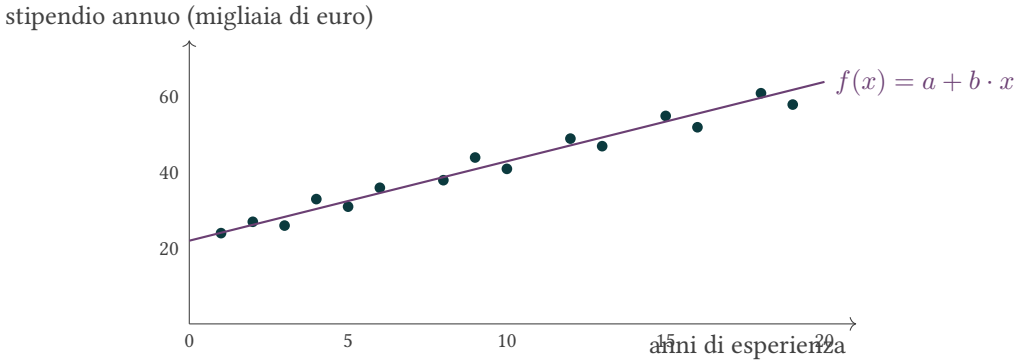


Figura 5.1. Predire lo stipendio dagli anni di esperienza: lo spazio delle funzioni è ristretto alle rette $f(x) = a + bx$, e il training cerca i parametri a e b che minimizzano l'errore medio sui campioni.

I parametri. Nell'esempio, $\mathcal{F}$ dipende da due **parametri**, a e b. Indichiamo con $\boldsymbol{\theta}$ l'insieme (di solito un vettore) di tutti i parametri di un modello, per cui $\mathcal{F} = \{f_{\boldsymbol{\theta}} \mid f_{\boldsymbol{\theta}}(x) = \theta_0 + \theta_1 \cdot x,\ \theta_0, \theta_1 \in \mathbb{R}\}$, e la nostra funzione è $f_{\boldsymbol{\theta}}$, dove $\boldsymbol{\theta}$ è il vettore che minimizza:

$$\boldsymbol{\theta} = \operatorname*{argmin}_{\boldsymbol{\theta}' \in \mathbb{R}^2} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}\big(f_{\boldsymbol{\theta}'}(x^{(i)}), y^{(i)}\big). \tag{5.3}$$

In generale preferiamo funzioni che si possano esprimere in forma analitica in funzione di alcuni parametri, e che siano **differenziabili**: in questo modo possiamo calcolarne il gradiente e

ottimizzarle con metodi numerici, come la discesa del gradiente stocastica. La buona notizia è che sommando e componendo funzioni differenziabili si ottengono ancora funzioni differenziabili, per cui possiamo combinarne quante ne vogliamo. A volte si scelgono funzioni con ulteriori proprietà, ad esempio convesse.

Training, validation e test. Come abbiamo già visto, un modello si costruisce in tre fasi. Si addestra su un **training set**, di solito grande (ad esempio "I'm happy" → Positive, "This is really bad" → Negative, "I don't like it" → Negative). Si regolano gli iperparametri sul **validation set** (o development set, o held-out set), come "This is good" → Positive o "What a shame" → Negative. Infine si misurano le prestazioni sul **test set**.

Classificazione. In un problema di classificazione il modello restituisce una **distribuzione di probabilità sulle classi**: ogni probabilità è compresa tra 0 e 1, e la somma delle probabilità di tutte le classi è 1. Ad esempio, data una frase, il modello restituisce la probabilità di Positivo, Negativo e Neutro. Perché una distribuzione di probabilità invece della sola classe finale? Perché in questo modo possiamo rendere f differenziabile.

5.2.1 Le componenti di un classificatore probabilistico

Dati m esempi di input e output $(x^{(i)}, y^{(i)})$, un classificatore probabilistico ha quattro componenti:

1. una **rappresentazione dell'input** tramite feature: per ogni osservazione $x^{(i)}$, un vettore di feature $[x_1, x_2, \ldots, x_n]$, dove la feature j dell'input $x^{(i)}$ si indica con x_j, con $x_j^{(i)}$ o a volte con $f_j(x^{(i)})$;
2. una **funzione di classificazione** che calcola $p(c \mid x)$, la stima della classe, ad esempio con la sigmoide o con il softmax;
3. una **funzione di loss** $\mathcal{L}$, ad esempio la cross-entropy loss;
4. un **algoritmo per ottimizzare** la funzione obiettivo, ad esempio la discesa del gradiente stocastica.

La regressione logistica ha quindi due fasi. Nel **training** impariamo i pesi $\mathbf{w}$ e il bias b con la discesa del gradiente stocastica e la cross-entropy loss. Nel **test**, dato un esempio x, calcoliamo $p(y \mid x)$ con i pesi appresi e restituiamo l'etichetta ($y = 1$ o $y = 0$) con la probabilità più alta.

5.3 Classificazione con la regressione logistica

Torniamo alla classificazione del testo: abbiamo un documento x e un insieme fissato di classi $C = \{c_1, \ldots, c_J\}$, e vogliamo predire una classe $\hat{y} \in C$. Nella **classificazione binaria**, data una serie di coppie di input e output $(x^{(i)}, y^{(i)})$, rappresentiamo ogni osservazione $x^{(i)}$ con un vettore di feature $[x_1, x_2, \ldots, x_n]$, e calcoliamo come output una classe predetta $\hat{y}^{(i)} \in \{0, 1\}$.

Per ogni feature x_i, il **peso** w_i dice quanto è importante quella feature. Ad esempio:

- x_i = "la recensione contiene *awesome*": $w_i = +10$;
- x_j = "la recensione contiene *abysmal*": $w_j = -10$;
- x_k = "la recensione contiene *mediocre*": $w_k = -2$.

Per un'osservazione $\mathbf{x} = [x_1, \ldots, x_n]$ abbiamo quindi un peso per ogni feature, $\mathbf{w} = [w_1, \ldots, w_n]$ (a volte i pesi si indicano con $\boldsymbol{\theta} = [\theta_1, \ldots, \theta_n]$), più un **bias** b. Per classificare sommiamo le feature pesate e il bias:

$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b = \mathbf{w} \cdot \mathbf{x} + b \tag{5.4}$$

e se la somma è alta diciamo $y = 1$, se è bassa $y = 0$.

5.3.1 La sigmoide

Vogliamo però un classificatore probabilistico, che come Naive Bayes ci dia una probabilità: un modello che sappia dirci $p(y = 1 \mid x; \boldsymbol{\theta})$ e $p(y = 0 \mid x; \boldsymbol{\theta})$. Il problema è che z non è una probabilità, è solo un numero, e può andare da $-\infty$ a $+\infty$. La soluzione è passare z in una funzione che vada da 0 a 1, la **sigmoide** (o funzione logistica, da cui il nome del modello):

$$y = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + \exp(-z)}. \tag{5.5}$$

La sigmoide è quasi lineare intorno a 0 e si appiattisce agli estremi, per cui tende a schiacciare i valori anomali verso 0 o 1, ed è differenziabile, il che sarà comodo per l'addestramento. Ha inoltre due proprietà che useremo:

$$\sigma(-z) = 1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}}, \qquad \sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big) \tag{5.6}$$

dove la prima segue dalla definizione (e dalla forma "simmetrica" della curva), la seconda dalle proprietà delle derivate[1].

L'idea della regressione logistica è quindi calcolare $\mathbf{w} \cdot \mathbf{x} + b$, passarlo nella sigmoide e trattare il risultato come una probabilità. Per avere una distribuzione basta assicurarsi che le due probabilità sommino a 1:

$$P(y = 1) = \sigma(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + \exp\big(-(\mathbf{w} \cdot \mathbf{x} + b)\big)} \tag{5.7}$$

$$P(y = 0) = 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) = 1 - \frac{1}{1 + \exp\big(-(\mathbf{w} \cdot \mathbf{x} + b)\big)} = \frac{\exp\big(-(\mathbf{w} \cdot \mathbf{x} + b)\big)}{1 + \exp\big(-(\mathbf{w} \cdot \mathbf{x} + b)\big)}. \tag{5.8}$$

Da probabilità a classificatore. Per decidere la classe di un'osservazione di test usiamo la soglia 0.5, detta **decision boundary**:

$$\hat{y} = \begin{cases} 1 & \text{se } P(y = 1 \mid x) > 0.5 \\ 0 & \text{altrimenti} \end{cases} \tag{5.9}$$

Dato che $\sigma(0) = 0.5$, la condizione equivale a $\mathbf{w} \cdot \mathbf{x} + b > 0$, e la classe è 0 se $\mathbf{w} \cdot \mathbf{x} + b \leq 0$.

[1] La quantità z si chiama anche **logit**, perché la funzione logit è l'inversa della sigmoide: $\text{logit}(p) = \sigma^{-1}(p) = \ln \frac{p}{1-p}$, ovvero il logaritmo degli odds. Chiamare z logit ricorda che lo stiamo interpretando come un log odds.

5.3.2 Un esempio di sentiment analysis

Vogliamo classificare la sentiment della seguente recensione:

> *It's hokey. There are virtually no surprises, and the writing is second-rate. So why was it so enjoyable? For one thing, the cast is great. Another nice touch is the music. I was overcome with the urge to get off the couch and start dancing. It sucked me in, and it'll do the same to you.*

Rappresentiamo la recensione con sei feature:

Var	Definizione	Valore
x_1	numero di parole del lessico positivo nel documento	3
x_2	numero di parole del lessico negativo nel documento	2
x_3	1 se "no" compare nel documento, 0 altrimenti	1
x_4	numero di pronomi di prima e seconda persona	3
x_5	1 se "!" compare nel documento, 0 altrimenti	0
x_6	logaritmo del numero di parole del documento	$\ln(66) = 4.19$

Le parole del lessico positivo sono *enjoyable*, *great* e *nice*, quelle del lessico negativo *hokey* e *second-rate*, e i pronomi sono *I*, *me* e *you*. Supponiamo di aver già imparato i pesi $\mathbf{w} = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7]$ e il bias $b = 0.1$. Il peso $w_1 = 2.5$ è positivo, mentre $w_2 = -5.0$ è negativo e circa doppio in valore assoluto: le parole negative sono associate negativamente alla sentiment positiva, e contano circa il doppio di quelle positive. La probabilità che la recensione sia positiva è:

$$\begin{aligned} p(+ \mid x) = P(y = 1 \mid x) &= \sigma(\mathbf{w} \cdot \mathbf{x} + b) \\ &= \sigma\big([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1\big) \\ &= \sigma(0.833) = 0.70 \end{aligned} \tag{5.10}$$

e quindi $p(- \mid x) = 1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b) = 0.30$: la recensione viene classificata come positiva (figura 5.2).

Feature per altri task. Possiamo costruire feature per la regressione logistica per qualsiasi task di classificazione. Nella **period disambiguation**, ad esempio, vogliamo decidere se un punto indica la fine di una frase ("This ends in a period.") o no ("The house at 465 Main St. is new."). Alcune feature possibili sono:

$$x_1 = \begin{cases} 1 & \text{se } Case(w_i) = \text{Lower} \\ 0 & \text{altrimenti} \end{cases} \qquad x_2 = \begin{cases} 1 & \text{se } w_i \in \text{AcronymDict} \\ 0 & \text{altrimenti} \end{cases}$$

$$x_3 = \begin{cases} 1 & \text{se } w_i = \text{St.} \wedge Case(w_{i-1}) = \text{Cap} \\ 0 & \text{altrimenti} \end{cases}$$

dove l'ultima combina due proprietà: un punto dopo *St.* preceduto da una parola maiuscola indica probabilmente l'abbreviazione di *street*, e non la fine della frase.

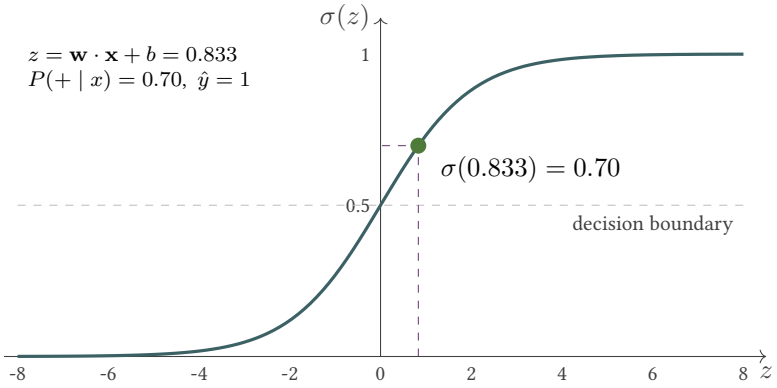


Figura 5.2. La sigmoide $\sigma(z) = 1/(1+e^{-z})$ e il punto corrispondente alla recensione dell'esempio: $z = 0.833$ è positivo, quindi $\sigma(z) = 0.70$ supera la decision boundary e la recensione è classificata come positiva.

Ricapitolando, nella regressione logistica binaria abbiamo un insieme di classi (sentiment positiva o negativa) e un vettore di feature $\mathbf{x} = [x_1, \ldots, x_n]$, ad esempio $x_1 = \text{count}(\text{awesome})$ e $x_2 = \log(\text{numero di parole})$. Abbiamo poi un vettore di pesi $\mathbf{w} = [w_1, \ldots, w_n]$, con un peso w_i per ogni feature, e un bias b. Con questi stimiamo $P(y = 1 \mid x) = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$.

5.4 La cross-entropy loss

Da dove vengono i pesi? Siamo nel caso supervisionato: per ogni x conosciamo l'etichetta corretta y (0 o 1), mentre il sistema produce una stima $\hat{y}$. Vogliamo scegliere $\mathbf{w}$ e b in modo da minimizzare la distanza tra la stima $\hat{y}^{(i)}$ e il valore vero $y^{(i)}$. Ci servono due cose:

- una misura di distanza, ovvero una **funzione di loss** (o funzione di costo): la **cross-entropy loss**;
- un **algoritmo di ottimizzazione** che aggiorni $\mathbf{w}$ e b per minimizzare la loss: la **discesa del gradiente stocastica**.

Vogliamo quindi sapere quanto l'output del classificatore, $\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, è lontano dall'output vero y, e chiamiamo questa differenza $L(\hat{y}, y)$.

L'intuizione. La cross-entropy loss è detta anche **negative log likelihood loss**, ed è un caso di **stima di massima verosimiglianza condizionale**: scegliamo i parametri $\mathbf{w}$ e b che massimizzano il logaritmo della probabilità delle etichette vere y nel training set, date le osservazioni x.

La derivazione per una singola osservazione. L'obiettivo è massimizzare la probabilità dell'etichetta corretta, $p(y \mid x)$. Dato che gli esiti possibili sono solo due (0 o 1), possiamo scrivere la probabilità prodotta dal classificatore come:

$$p(y \mid x) = \hat{y}^{\,y} \, (1 - \hat{y})^{1-y} \tag{5.11}$$

che per $y = 1$ diventa $\hat{y}$, e per $y = 0$ diventa $1 - \hat{y}$. Prendiamo il logaritmo di entrambi i membri, che è comodo matematicamente e non cambia niente, perché i valori che massimizzano $\log p(y \mid x)$ massimizzano anche $p(y \mid x)$:

$$\log p(y \mid x) = \log \left[\hat{y}^{\,y} (1 - \hat{y})^{1-y}\right] = y \log \hat{y} + (1 - y) \log(1 - \hat{y}). \tag{5.12}$$

Infine cambiamo segno, per trasformare la quantità da massimizzare in una loss da minimizzare. Il risultato è la **cross-entropy loss** (si chiama così perché è la formula della cross-entropy tra y e $\hat{y}$):

$$L_{CE}(\hat{y}, y) = -\log p(y \mid x) = -\big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\big] \tag{5.13}$$

e sostituendo la definizione di $\hat{y}$:

$$L_{CE}(\hat{y}, y) = -\big[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1 - y) \log\big(1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b)\big)\big]. \tag{5.14}$$

Funziona? Torniamo all'esempio della recensione. Vogliamo che la loss sia più piccola se la stima del modello è vicina a quella corretta, e più grande se il modello si confonde.

Supponiamo prima che l'etichetta vera sia $y = 1$ (recensione positiva): il modello se la cava bene, perché le ha assegnato 0.70. La loss è:

$$L_{CE}(\hat{y}, y) = -\big[\log \sigma(\mathbf{w} \cdot \mathbf{x} + b)\big] = -\log(0.70) = 0.36.$$

Se invece l'etichetta vera fosse $y = 0$ (magari la recensione continuava con "But bottom line, the movie is terrible! I beg you not to see it!"), il modello si sarebbe confuso, e la loss è:

$$L_{CE}(\hat{y}, y) = -\big[\log\big(1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b)\big)\big] = -\log(0.30) = 1.2.$$

La loss è quindi più grande quando il modello sbaglia, come volevamo. In generale $-\log$ va da 0 (il logaritmo di 1, nessuna perdita) a infinito (il logaritmo di 0), e dato che le due probabilità sommano a 1, ogni aumento della probabilità della risposta corretta avviene a spese di quella sbagliata.

5.5 Discesa del gradiente

Rendiamo esplicito che la loss dipende dai parametri $\boldsymbol{\theta} = (w_1, w_2, \ldots, w_n, b)$, e scriviamo $\hat{y}$ come $f(x; \boldsymbol{\theta})$. Vogliamo i pesi che minimizzano la loss media su tutti gli esempi:

$$\hat{\boldsymbol{\theta}} = \operatorname*{argmin}_{\boldsymbol{\theta}} \frac{1}{m} \sum_{i=1}^{m} L_{CE}\big(f(x^{(i)}; \boldsymbol{\theta}), y^{(i)}\big). \tag{5.15}$$

L'intuizione della **discesa del gradiente** è questa: come arriviamo in fondo a un canyon, dove scorre il fiume? Ci guardiamo intorno a 360 gradi, troviamo la direzione in cui il terreno scende più ripido, e andiamo da quella parte. Per la regressione logistica la loss è **convessa**: una funzione convessa ha un solo minimo, e la discesa del gradiente, da qualunque punto parta, è garantita trovarlo[2].

[2] La loss delle reti neurali, invece, non è convessa.

Un solo parametro. Visualizziamo prima il caso di un singolo parametro scalare w. Dato il valore corrente w_1, dobbiamo farlo diventare più grande o più piccolo? La risposta è spostarsi nel verso opposto alla pendenza della funzione: se la pendenza della loss in w_1 è negativa, ci spostiamo verso destra (figura 5.3). Più in generale, il **gradiente** di una funzione di più variabili è un vettore che punta nella direzione di massima crescita della funzione. La discesa del gradiente calcola il gradiente della loss nel punto corrente e si sposta nella direzione **opposta**.

Di quanto ci spostiamo? Del valore del gradiente (la pendenza, nel nostro esempio a una variabile), $\frac{d}{dw}L(f(x;w),y)$, pesato da un **learning rate** η:

$$w_{t+1} = w_t - \eta \frac{d}{dw} L\big(f(x;w),y\big) \tag{5.16}$$

dove un learning rate più alto vuol dire spostare w più velocemente.

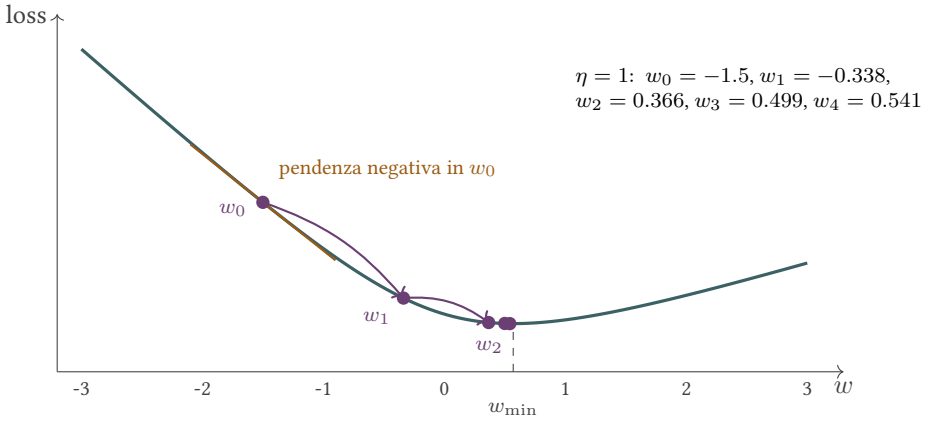


Figura 5.3. La discesa del gradiente con un solo peso w, sulla cross-entropy media di un piccolo training set (bias fissato a 0). In $w_0 = -1.5$ la pendenza è negativa, quindi ci si sposta verso destra. Con $\eta = 1$ i passi si accorciano man mano che la pendenza diminuisce, e w si avvicina al minimo $w_{\min} \approx 0.57$.

N dimensioni. In realtà non vogliamo solo sapere se spostarci a destra o a sinistra, ma in quale direzione muoverci nello spazio a N dimensioni degli N parametri che compongono $\boldsymbol{\theta}$. Il gradiente è proprio un vettore di questo tipo, con una componente per ognuna delle N dimensioni. Con due sole dimensioni, ad esempio un peso w e un bias b, la loss $\text{Cost}(w,b)$ è una superficie a forma di scodella. L'opposto del gradiente in un punto è allora un vettore nel piano (w,b) che indica dove scendere.

I gradienti reali sono molto più lunghi, perché i pesi sono moltissimi. Per ogni dimensione w_i, la componente del gradiente ci dice la pendenza rispetto a quella variabile, ovvero quanto una piccola variazione di w_i influenza la loss totale L, e si esprime con la derivata parziale $\frac{\partial}{\partial w_i}$ della loss. Il gradiente è il vettore di queste derivate parziali:

$$\nabla L\big(f(x;\boldsymbol{\theta}),y\big) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x;\boldsymbol{\theta}),y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x;\boldsymbol{\theta}),y) \\ \frac{\partial}{\partial b} L(f(x;\boldsymbol{\theta}),y) \end{bmatrix} \qquad \text{e} \qquad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla L\big(f(x;\boldsymbol{\theta}),y\big). \tag{5.17}$$

5.5.1 Il gradiente della regressione logistica

Le derivate parziali della cross-entropy loss (5.14) hanno una forma molto elegante:

$$\frac{\partial L_{CE}(\hat{y}, y)}{\partial w_j} = \big[\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y\big]\, x_j, \qquad \frac{\partial L_{CE}(\hat{y}, y)}{\partial b} = \sigma(\mathbf{w} \cdot \mathbf{x} + b) - y. \tag{5.18}$$

Il gradiente rispetto a un singolo peso w_j è un valore molto intuitivo: la differenza tra la stima $\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$ e il valore vero y, moltiplicata per il valore della feature x_j corrispondente.

Osservazione (Da dove viene il gradiente). La (5.18) si ricava con la regola della catena, usando $\frac{d}{dx} \ln x = \frac{1}{x}$ e la proprietà $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Posto $\hat{y} = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$, e osservando che $\frac{\partial}{\partial w_j}(\mathbf{w} \cdot \mathbf{x} + b) = x_j$:

$$\frac{\partial L_{CE}}{\partial w_j} = -\left[\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}}\right] \hat{y}(1 - \hat{y})\, x_j = -\big[y(1 - \hat{y}) - (1 - y)\hat{y}\big]\, x_j = (\hat{y} - y)\, x_j.$$

5.5.2 La discesa del gradiente stocastica

La **discesa del gradiente stocastica** (stochastic gradient descent, SGD) è un algoritmo online: minimizza la loss calcolandone il gradiente dopo ogni esempio di training, e sposta $\boldsymbol{\theta}$ nella direzione giusta, ovvero opposta al gradiente (algoritmo 4). Si chiama stocastica perché sceglie a caso un esempio alla volta.

Algoritmo 4 La discesa del gradiente stocastica: L è la funzione di loss, f una funzione parametrizzata da $\boldsymbol{\theta}$, x e y gli input e le etichette del training set.

```
function StochasticGradientDescent(L(), f(), x, y)
    θ ← 0
    repeat
        for all coppia di training (x^(i), y^(i)), in ordine casuale do
            calcola ŷ^(i) = f(x^(i); θ) e la loss L(ŷ^(i), y^(i))   ▷ facoltativo: come stiamo andando?
            g ← ∇_θ L(f(x^(i); θ), y^(i))                          ▷ come spostare θ per aumentare la loss?
            θ ← θ − η g                                            ▷ andiamo dalla parte opposta
        end for
    until convergenza
    return θ
end function
```

Iperparametri. Il learning rate η è un **iperparametro**: se è troppo alto, i passi sono troppo lunghi e l'algoritmo supera il minimo, mentre se è troppo basso i passi sono troppo corti e l'algoritmo impiega troppo tempo. Gli iperparametri sono, in breve, un tipo speciale di parametro di un modello di machine learning: invece di essere appresi dall'algoritmo a partire dalla supervisione, come i parametri normali, vengono scelti da chi progetta l'algoritmo. Nella versione web, la figura 5.3 permette di provare diversi valori di η.

Un esempio. Facciamo un passo di discesa del gradiente su un esempio di sentiment in miniatura, con etichetta vera $y = 1$ (positiva) e due feature: $x_1 = 3$ (numero di parole del lessico positivo) e

$x_2 = 2$ (numero di parole del lessico negativo). I tre parametri in $\boldsymbol{\theta}_0$ (due pesi e un bias) partono da zero, $w_1 = w_2 = b = 0$, e il learning rate è $\eta = 0.1$.

Il passo di aggiornamento è $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla L(f(x; \boldsymbol{\theta}), y)$, dove il gradiente ha tre componenti. Con la (5.18), e dato che $\sigma(0) = 0.5$:

$$\nabla_{w,b} L = \begin{bmatrix} \frac{\partial L_{CE}(\hat{y},y)}{\partial w_1} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial w_2} \\ \frac{\partial L_{CE}(\hat{y},y)}{\partial b} \end{bmatrix} = \begin{bmatrix} (\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y)\, x_1 \\ (\sigma(\mathbf{w} \cdot \mathbf{x} + b) - y)\, x_2 \\ \sigma(\mathbf{w} \cdot \mathbf{x} + b) - y \end{bmatrix} = \begin{bmatrix} (\sigma(0) - 1)\, x_1 \\ (\sigma(0) - 1)\, x_2 \\ \sigma(0) - 1 \end{bmatrix} = \begin{bmatrix} -0.5\, x_1 \\ -0.5\, x_2 \\ -0.5 \end{bmatrix} = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}.$$

Ci spostiamo quindi nella direzione opposta al gradiente:

$$\boldsymbol{\theta}_1 = \begin{bmatrix} w_1 \\ w_2 \\ b \end{bmatrix} - \eta \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 0.15 \\ 0.1 \\ 0.05 \end{bmatrix}.$$

Dopo un passo, tutti i pesi sono cresciuti, perché l'esempio era positivo. Osserviamo che, vedendo abbastanza esempi negativi con molte parole del lessico negativo, w_2 diventerebbe prima o poi negativo.

5.5.3 Mini-batch

La discesa del gradiente stocastica sceglie un solo esempio a caso alla volta, e questo può portare a movimenti molto irregolari. È più comune calcolare il gradiente su gruppi (*batch*) di esempi di training. Nel **batch training** si usa l'intero dataset, nel **mini-batch training** un gruppo di m esempi (ad esempio 512 o 1024). In questo caso il costo è la loss media sugli esempi del mini-batch, e il gradiente è la media dei gradienti dei singoli esempi:

$$\frac{\partial \,\text{Cost}(\hat{y}, y)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^{m} \left[\sigma(\mathbf{w} \cdot \mathbf{x}^{(i)} + b) - y^{(i)}\right] x_j^{(i)}. \tag{5.19}$$

5.6 Overfitting e regolarizzazione

Un modello che si adatta perfettamente ai dati di training ha un problema: va in **overfitting**, ovvero modella anche il rumore. Una parola casuale che predice perfettamente y, perché per caso compare in una sola classe, riceverà un peso altissimo, e il modello non saprà generalizzare su un test set in cui quella parola non c'è. Un buon modello deve invece saper generalizzare.

Consideriamo due recensioni di training: "This movie drew me in, and it'll do the same to you." (+) e "I can't tell you how much I hated this movie. It sucked." (−). Feature come $x_1 =$ this, $x_2 =$ movie e $x_3 =$ hated sono utili o almeno innocue. Feature basate su 4-grammi, come $x_4 =$ drew me in, $x_5 =$ the same to you o $x_7 =$ tell you how much, si limitano invece a "memorizzare" il training set, e possono causare problemi.

Un modello a 4-grammi su pochi dati memorizza i dati: ha un'accuratezza del 100% sul training set, ma sarà sorpreso dai 4-grammi nuovi del test set, e avrà un'accuratezza bassa. I modelli troppo potenti possono quindi andare in overfitting, adattandosi così bene ai dettagli dei dati di training da non generalizzare sul test set. Per evitarlo si usa la **regolarizzazione** nella regressione logistica e il **dropout** nelle reti neurali.

Regolarizzazione. L'idea è aggiungere alla funzione obiettivo un termine di regolarizzazione $R(\boldsymbol{\theta})$ che penalizza i pesi grandi: adattarsi bene ai dati con tanti pesi grandi è peggio che adattarsi un po' meno bene con pesi piccoli. Scrivendo l'obiettivo come massimizzazione della log probability (invece che come minimizzazione della loss):

$$\hat{\boldsymbol{\theta}} = \operatorname*{argmax}_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log P(y^{(i)} \mid x^{(i)}) - \alpha R(\boldsymbol{\theta}) \tag{5.20}$$

dove α regola la forza della regolarizzazione. Le scelte più comuni per $R(\boldsymbol{\theta})$ sono due:

- **Regolarizzazione L2** (ridge regression): $R(\boldsymbol{\theta})$ è la somma dei quadrati dei pesi. Si chiama così perché è il quadrato della norma L2 $\|\boldsymbol{\theta}\|_2$, ovvero della distanza euclidea di $\boldsymbol{\theta}$ dall'origine:

$$R(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_2^2 = \sum_{j=1}^{n} \theta_j^2 \qquad \Longrightarrow \qquad \hat{\boldsymbol{\theta}} = \operatorname*{argmax}_{\boldsymbol{\theta}} \left[\sum_{i=1}^{m} \log P(y^{(i)} \mid x^{(i)}) \right] - \alpha \sum_{j=1}^{n} \theta_j^2. \tag{5.21}$$

- **Regolarizzazione L1** (lasso regression): $R(\boldsymbol{\theta})$ è la somma dei valori assoluti dei pesi, ovvero la norma L1 $\|\boldsymbol{\theta}\|_1$ (la distanza di Manhattan):

$$R(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_{i=1}^{n} |\theta_i| \qquad \Longrightarrow \qquad \hat{\boldsymbol{\theta}} = \operatorname*{argmax}_{\boldsymbol{\theta}} \left[\sum_{i=1}^{m} \log P(y^{(i)} \mid x^{(i)}) \right] - \alpha \sum_{j=1}^{n} |\theta_j|. \tag{5.22}$$

La L2 è più facile da ottimizzare, perché la derivata di θ^2 è semplicemente 2θ, e preferisce vettori con tanti pesi piccoli. La L1 invece preferisce soluzioni sparse, con alcuni pesi grandi e molti pesi a zero, e quindi con molte meno feature. In entrambi i casi il bias di solito non viene regolarizzato.

5.7 Regressione logistica multinomiale

Spesso servono più di due classi: sentiment positiva, negativa o neutra, le parti del discorso (nome, verbo, aggettivo, avverbio, preposizione, ...), o le classi di intervento a cui smistare gli SMS di emergenza. Con più di due classi si usa la **regressione logistica multinomiale**, detta anche **softmax regression** o multinomial logit (e, in passato, maximum entropy modeling o MaxEnt). Da qui in poi, "regressione logistica" indicherà il caso binario.

5.7.1 Il softmax

Le probabilità di tutte le classi devono ancora sommare a 1, ad esempio $P(\text{positive} \mid \text{doc}) + P(\text{negative} \mid \text{doc}) + P(\text{neutral} \mid \text{doc}) = 1$. Serve quindi una generalizzazione della sigmoide, il **softmax**, che prende un vettore $\mathbf{z} = [z_1, z_2, \ldots, z_k]$ di k valori arbitrari e restituisce una distribuzione di probabilità, con ogni valore in $[0, 1]$ e tutti i valori che sommano a 1:

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^{k} \exp(z_j)}, \qquad 1 \leq i \leq k. \tag{5.23}$$

Il softmax di un vettore è quindi a sua volta un vettore:

$$\text{softmax}(\mathbf{z}) = \left[\frac{\exp(z_1)}{\sum_{i=1}^{k} \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^{k} \exp(z_i)}, \ldots, \frac{\exp(z_k)}{\sum_{i=1}^{k} \exp(z_i)} \right] \tag{5.24}$$

dove il denominatore $\sum_{i=1}^{k} e^{z_i}$ serve a normalizzare tutti i valori in probabilità. Ad esempio, dato il vettore $\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$, si ottiene

$$\text{softmax}(\mathbf{z}) = [0.055, 0.090, 0.0067, 0.10, 0.74, 0.010].$$

Come la sigmoide, il softmax schiaccia i valori verso 0 o 1: se uno degli input è più grande degli altri, la sua probabilità tende a 1, e le probabilità degli input più piccoli vengono soppresse (figura 5.4).

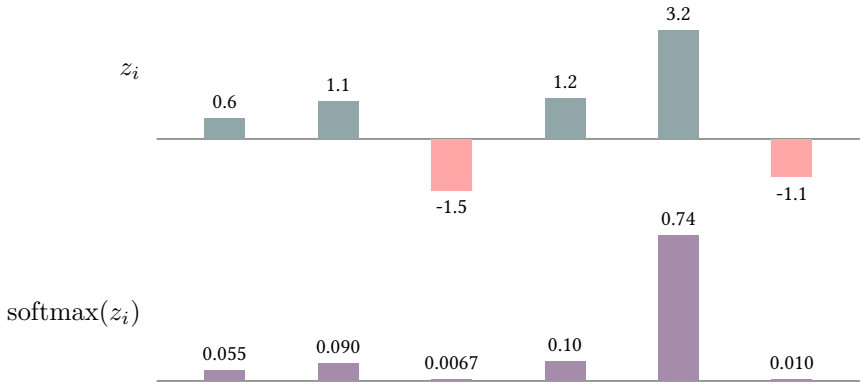


Figura 5.4. Il softmax del vettore $\mathbf{z} = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$: in alto i logit, in basso le probabilità. Il logit più grande, 3.2, si prende da solo il 74% della massa di probabilità.

5.7.2 Il softmax nella regressione logistica multinomiale

Come per la sigmoide, l'input del softmax è il prodotto scalare tra un vettore di pesi e il vettore di input, più un bias. Ora però servono un vettore di pesi $\mathbf{w}_c$ e un bias b_c separati per ognuna delle k classi:

$$p(y = c \mid x) = \frac{\exp(\mathbf{w}_c \cdot \mathbf{x} + b_c)}{\sum_{j=1}^{k} \exp(\mathbf{w}_j \cdot \mathbf{x} + b_j)}. \tag{5.25}$$

Mettendo i k vettori di pesi come righe di una matrice W e i bias in un vettore $\mathbf{b}$, tutte le probabilità si calcolano in un colpo solo con $\hat{\mathbf{y}} = \text{softmax}(W\mathbf{x} + \mathbf{b})$.

Feature nel caso binario e multinomiale. Nella regressione logistica binaria un peso positivo spinge verso $y = 1$ e un peso negativo verso $y = 0$. Consideriamo ad esempio la feature

$$x_5 = \begin{cases} 1 & \text{se “!” } \in \text{ doc} \\ 0 & \text{altrimenti} \end{cases}$$

con peso $w_5 = 3.0$, che spinge verso la sentiment positiva. Nel caso multinomiale, invece, ogni classe ha i suoi pesi, e la stessa feature può essere un indizio a favore o contro ciascuna classe. Con tre classi ($+$, $-$ e 0, neutra) potremmo avere $w_{5,+} = 3.5$, $w_{5,-} = 3.1$ e $w_{5,0} = -5.3$: il punto esclamativo è un indizio sia per la sentiment positiva sia per quella negativa, ma è un forte indizio contro la neutralità.

5.8 La classificazione in pratica

In Python, il pacchetto **scikit-learn** fornisce tutto quello che abbiamo visto, insieme a strumenti per estrarre le feature e calcolare le metriche della sezione 4.8:

- `SGDClassifier(loss='log_loss')`: regressione logistica addestrata con la discesa del gradiente stocastica;
- `LogisticRegression()`: regressione logistica addestrata con altri algoritmi di ottimizzazione, non con SGD;
- `MultinomialNB()`: Naive Bayes multinomiale.

Bibliografia

[1] Laura Banarescu et al. “Abstract Meaning Representation for Sembanking”. In: *Proceedings of the 7th Linguistic Annotation Workshop and Interoperability with Discourse*. 2013, pp. 178–186.

[2] Emily M. Bender e Batya Friedman. “Data Statements for Natural Language Processing: Toward Mitigating System Bias and Enabling Better Science”. In: *Transactions of the Association for Computational Linguistics* 6 (2018), pp. 587–604.

[3] Taylor Berg-Kirkpatrick, David Burkett e Dan Klein. “An Empirical Investigation of Statistical Significance in NLP”. In: *Proceedings of EMNLP 2012*. 2012, pp. 995–1005.

[4] Steven Bird, Ewan Klein e Edward Loper. *Natural Language Processing with Python*. O’Reilly, 2009.

[5] Thorsten Brants et al. “Large Language Models in Machine Translation”. In: *Proceedings of EMNLP-CoNLL 2007*. 2007, pp. 858–867.

[6] Stanley F. Chen e Joshua Goodman. *An Empirical Study of Smoothing Techniques for Language Modeling*. Rapp. tecn. TR-10-98. Computer Science Group, Harvard University, 1998.

[7] Kenneth W. Church e William A. Gale. “A Comparison of the Enhanced Good-Turing and Deleted Estimation Methods for Estimating Probabilities of English Bigrams”. In: *Computer Speech and Language* 5 (1991), pp. 19–54.

[8] Bradley Efron e Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, 1993.

[9] Tommaso Fornaciari et al. “Hard and Soft Evaluation of NLP Models with BOOtSTrap SAmpling – BooStSa”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. 2022, pp. 127–134.

[10] Timnit Gebru et al. *Datasheets for Datasets*. arXiv:1803.09010. 2020.

[11] Daniel Jurafsky e James H. Martin. *Speech and Language Processing*. 3rd edition draft, release del 6 gennaio. 2026. URL: https://web.stanford.edu/~jurafsky/slp3/.

[12] Svetlana Kiritchenko e Saif M. Mohammad. “Examining Gender and Race Bias in Two Hundred Sentiment Analysis Systems”. In: *Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics (*SEM)*. 2018, pp. 43–53.

[13] Reinhard Kneser e Hermann Ney. “Improved Backing-off for M-gram Language Modeling”. In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Vol. 1. 1995, pp. 181–184.

[14] Taku Kudo. “Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates”. In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics*. 2018, pp. 66–75.

[15] Tomas Mikolov et al. *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781. 2013.

[16] Margaret Mitchell et al. “Model Cards for Model Reporting”. In: *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*. 2019, pp. 220–229.

[17] Frederick Mosteller e David L. Wallace. “Inference in an Authorship Problem”. In: *Journal of the American Statistical Association* 58.302 (1963), pp. 275–309.

[18] Bo Pang, Lillian Lee e Shivakumar Vaithyanathan. “Thumbs up? Sentiment Classification using Machine Learning Techniques”. In: *Proceedings of EMNLP 2002*. 2002, pp. 79–86.

[19] Martin F. Porter. “An Algorithm for Suffix Stripping”. In: *Program* 14.3 (1980), pp. 130–137.

[20] Klaus R. Scherer. “Psychological Models of Emotion”. In: *The Neuropsychology of Emotion*. A cura di Joan C. Borod. Oxford University Press, 2000, pp. 137–162.

[21] Mike Schuster e Kaisuke Nakajima. “Japanese and Korean Voice Search”. In: *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. 2012, pp. 5149–5152.

[22] Rico Sennrich, Barry Haddow e Alexandra Birch. “Neural Machine Translation of Rare Words with Subword Units”. In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*. 2016, pp. 1715–1725.

[23] Claude E. Shannon. “A Mathematical Theory of Communication”. In: *Bell System Technical Journal* 27.3 (1948), pp. 379–423.

[24] Philip J. Stone et al. *The General Inquirer: A Computer Approach to Content Analysis*. MIT Press, 1966.

[25] Ashish Vaswani et al. “Attention Is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[26] Joseph Weizenbaum. “ELIZA – A Computer Program for the Study of Natural Language Communication between Man and Machine”. In: *Communications of the ACM* 9.1 (1966), pp. 36–45.

[27] Theresa Wilson, Janyce Wiebe e Paul Hoffmann. “Recognizing Contextual Polarity in Phrase-Level Sentiment Analysis”. In: *Proceedings of HLT-EMNLP 2005*. 2005, pp. 347–354.

[28] Shunyu Yao et al. *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629. 2022.